

PROGRAMME D'INTRODUCTION
À L'INTELLIGENCE ARTIFICIELLE (PIIA)

Spécial Machine Learning • 2^e Cohorte • Promotion 2026

MATHÉMATIQUES FONDAMENTALES POUR IA

*Calcul numérique | Algèbre linéaire | Probabilités
Statistiques | Mathématiques discrètes*

*« Si les gens ne croient pas que les mathématiques sont
simples, c'est uniquement parce qu'ils ne réalisent pas à quel point la
vie est compliquée. »*

— John von Neumann —

Liste des notations

Notation	Définition
Algèbre et Espaces	
$\mathbb{R}$	Ensemble des nombres réels
$\mathbb{R}^n$	Espace vectoriel réel à n dimensions
$\mathbf{x}, \mathbf{y}$	Vecteurs (écrits en minuscules grasses)
A, X, W	Matrices (écrites en majuscules)
$A^\top$	Transposée de la matrice A
A^{-1}	Inverse de la matrice A
$\text{tr}(A)$	Trace de la matrice A
$\text{rg}(A)$	Rang de la matrice A
$\ \mathbf{x}\ $	Norme euclidienne (ou norme L_2) d'un vecteur
$\kappa(A)$	Nombre de conditionnement d'une matrice A
Calcul Différentiel et Optimisation	
$\nabla f(\mathbf{x})$	Gradient de la fonction scalaire f évalué au point $\mathbf{x}$
$\nabla^2 f(\mathbf{x})$	Matrice hessienne de la fonction f évaluée au point $\mathbf{x}$
$J_f(\mathbf{x})$	Matrice jacobienne de la fonction vectorielle f
$\frac{\partial f}{\partial x_i}$	Dérivée partielle de f par rapport à la variable x_i
$\mathcal{L}(\theta)$	Fonction de coût (ou de perte) paramétrée par θ
α	Taux d'apprentissage (<i>learning rate</i>)

Liste des sigles et abréviations

Sigle	Signification
IA	Intelligence Artificielle
ML	Machine Learning (Apprentissage Automatique)
DL	Deep Learning (Apprentissage Profond)
ACP	Analyse en Composantes Principales
SVD	Singular Value Decomposition (Décomposition en Valeurs Singulières)
RNN	Recurrent Neural Network (Réseau de Neurones Récurrent)
LSTM	Long Short-Term Memory (Mémoire à Long Court Terme)
LLM	Large Language Model (Grand Modèle de Langage)
MSE	Mean Squared Error (Erreur Quadratique Moyenne)
GPU	Graphics Processing Unit (Processeur Graphique)
LoRA	Low-Rank Adaptation (Adaptation de Rang Faible)

Table des figures

1.1	La fonction sigmoïde et sa dérivée. On observe que la dérivée est maximale en $x = 0$ (valeur 0.25) et tend rapidement vers 0 pour $ x $ grand. C'est la cause du problème d'évanouissement du gradient dans les réseaux profonds.	4
1.2	Lignes de niveau d'une fonction de coût convexe quadratique. Les vecteurs rouges représentent le gradient ∇f . On remarque géométriquement qu'ils sont toujours orthogonaux aux lignes de niveau et pointent vers l'extérieur (direction de plus forte croissance).	7
1.3	Un point-selle (ou col) en $(0,0)$. Le gradient y est nul (le terrain est localement plat), mais ce n'est ni un minimum ni un maximum. L'analyse des valeurs propres de la matrice hessienne est indispensable pour identifier cette géométrie (ici, une valeur propre positive et une négative).	11
1.4	Trajectoire des poids lors de la descente de gradient. Si le pas d'apprentissage α est bien calibré (vert), l'algorithme converge efficacement. S'il est trop grand (rouge), l'algorithme "rebondit" sur les parois de la vallée d'erreur (oscillations), ce qui ralentit considérablement la convergence, voire provoque une divergence.	13
2.1	Interprétation géométrique du produit matriciel. Une matrice $A \in \mathbb{R}^{2 \times 2}$ appliquée à une grille de points dans $\mathbb{R}^2$ agit comme une déformation de l'espace (ici une combinaison de rotation et d'étirement). L'aire d'un carré unitaire formé par les vecteurs de base devient un parallélogramme dont la surface est exactement égale au déterminant de A	19
2.2	Application du Théorème Spectral à une matrice de covariance. Le nuage de points génère une matrice symétrique définie positive. La diagonalisation révèle les axes naturels de la distribution : les vecteurs propres. La longueur de l'axe principal (en rouge) est proportionnelle à la racine de la valeur propre associée λ_1 , justifiant mathématiquement le principe de l'Analyse en Composantes Principales (ACP).	23
2.3	Illustration du Théorème d'Eckart-Young via la Décomposition en Valeurs Singulières (SVD). Une matrice (ici représentée par une carte de chaleur de 100×100) de rang plein peut être approchée avec une fidélité étonnante en ne conservant qu'une fraction de ses valeurs singulières. Avec un rang $k = 20$, 99% de la variance structurelle est préservée pour un stockage drastiquement réduit.	24
2.4	Impact du conditionnement matriciel κ sur l'optimisation. À gauche : la matrice Hessienne est l'identité ($\kappa = 1$), les lignes de niveau sont parfaitement circulaires et l'algorithme converge en ligne droite. À droite : $\kappa = 15$ crée un "ravin". Le gradient (qui est orthogonal aux lignes de niveau elliptiques) pointe presque exclusivement vers les murs et très peu vers le minimum. L'algorithme rebondit violemment.	27

3.1	Transformation des scores bruts (Logits) d'un réseau de neurones en probabilités via la fonction Softmax. On remarque que la Softmax garantit l'Axiome de Kolmogorov (K1) : toutes les valeurs sont positives et leur somme vaut exactement 1. De plus, elle amplifie les différences (la classe "Voiture" domine largement la distribution finale).	30
3.2	L'Inférence Bayésienne en action. Avant de voir les données, notre croyance (Prior, en pointillés) est centrée sur 3. Les nouvelles observations (Vraisemblance, en orange) pointent fortement vers 7. Le Théorème de Bayes fusionne ces deux informations : la croyance mise à jour (Posterior, en bleu) est un compromis centré autour de 6. La variance du Posterior est plus faible, ce qui signifie que notre incertitude a diminué grâce à l'apprentissage.	31
3.3	Simulation de la Loi des Grands Nombres. Bien que les premiers lancers d'un dé soient soumis à un fort chaos aléatoire (variance élevée), la moyenne empirique converge inexorablement vers l'espérance mathématique théorique de 3.5 à mesure que la taille de l'échantillon N augmente. C'est ce théorème qui garantit que l'apprentissage sur un grand jeu de données (<i>Risque Empirique</i>) généralisera bien dans le monde réel (<i>Risque Vrai</i>).	33
3.4	Le Théorème Central Limite. L'histogramme empirique (en gris) montre le résultat de l'addition de seulement 12 variables aléatoires de loi Uniforme (qui n'ont pas du tout une forme de cloche). Magiquement, leur somme épouse presque parfaitement la courbe théorique de la loi Normale (en rouge). Ce phénomène justifie l'utilisation systématique de la loi Gaussienne pour modéliser le bruit des capteurs physiques ou initialiser les poids des réseaux de neurones profonds.	34
4.1	Sensibilité aux anomalies (<i>outliers</i>). L'ajout d'une seule valeur aberrante extrême (ici 120) tire violemment la moyenne vers la droite. En revanche, la médiane reste parfaitement stable au centre de la distribution principale. C'est la raison géométrique pour laquelle la fonction de perte absolue (MAE, optimisant la médiane) est plus robuste que la perte quadratique (MSE, optimisant la moyenne) en <i>Machine Learning</i>	40
4.2	L'Analyse Exploratoire des Données (EDA). À gauche : L'estimation par noyau de densité (KDE) révèle une bimodalité (deux bosses). Cela indique formellement qu'une variable cachée sépare les données en deux sous-populations (ici, la taille des femmes et des hommes). À droite : Le Boxplot offre un résumé extrêmement compact permettant d'identifier immédiatement la médiane et les potentiels <i>outliers</i> (les points au-delà des moustaches).	42
4.3	Le piège statistique des tests multiples (<i>P-Hacking</i>). Sur 1 000 tests A/B comparant deux modèles générés de manière <i>purement aléatoire</i> (l'hypothèse nulle H_0 est donc strictement vraie), environ 5% des tests (en rouge) tombent naturellement sous le seuil $\alpha = 0.05$. Si l'on teste aveuglément des dizaines d'hyperparamètres, on finira par sélectionner une "fausse découverte" statistique. La correction de Bonferroni est donc impérative.	45

4.4	Le Paradoxe de Simpson et la Variable Confondante. À gauche (Vision IA actuelle) : Si l'on ne regarde que la corrélation globale, l'IA en conclut que plus on augmente le traitement, plus le taux de guérison grimpe (Tendance positive rouge). À droite (Inférence Causale) : Si l'on sépare la population selon la variable confondante (La gravité de la maladie d'origine), on observe que la vraie tendance causale est <i>négative</i> pour chaque sous-groupe. L'IA naïve aurait prescrit un traitement mortel.	47
5.1	Classification topologique des fonctions en <i>Machine Learning</i> . (1) Une fonction Injective ne superpose jamais deux entrées (aucune collision), mais peut ignorer une partie de l'espace d'arrivée. (2) Une fonction Surjective génère tous les résultats possibles, mais détruit de l'information (réduction de dimension des réseaux convolutifs). (3) Une fonction Bijective relie chaque entrée à une sortie unique et est strictement inversible. Les modèles génératifs de pointe (<i>Normalizing Flows</i>) exploitent cette bijection pour mapper parfaitement l'espace du bruit (loi normale) vers l'espace des données (images réelles).	52
5.2	La Malédiction de la Dimensionnalité (<i>Curse of Dimensionality</i>). En combinatoire, si l'on effectue une recherche en grille (<i>Grid Search</i>) en testant 5 valeurs possibles pour n hyperparamètres, le nombre de modèles à entraîner s'envole de manière exponentielle (5^n). Au-delà de 4 paramètres, le calcul exhaustif devient physiquement intraitable, justifiant l'usage de méthodes stochastiques (Recherche aléatoire) ou probabilistes (Optimisation Bayésienne).	53
5.3	Modélisation d'une équation mathématique sous forme de Graphe Orienté Acyclique (DAG) . C'est exactement la structure de données discrète générée dynamiquement par PyTorch lors de l'entraînement. La passe avant (<i>Forward Pass</i>) correspond à un tri topologique du graphe de la gauche vers la droite. Lors de la <i>rétropropagation</i> , l'algorithme parcourt ce graphe à l'envers (DFS) pour dériver la fonction de perte en appliquant la règle de la chaîne à chaque arête.	55
5.4	Comparaison des complexités algorithmiques (Grand O) face à l'allongement du contexte en Intelligence Artificielle. Le mécanisme d' <i>Attention</i> des Transformers exige que chaque mot se compare à tous les autres, générant une complexité spatio-temporelle en $\mathcal{O}(N^2)$ (en rouge). Cette courbe heurte très rapidement la limite physique de la VRAM des processeurs graphiques (GPU). La recherche de modèles sub-quadratiques (Mamba en $\mathcal{O}(N)$) est l'enjeu majeur pour traiter des livres entiers ou de l'ADN.	56

Liste des tableaux

1.1	Dérivées usuelles, dont deux fonctions d'activation classiques en apprentissage profond (sigmoïde, tangente hyperbolique).	3
3.1	Espérance et variance des lois usuelles.	34

Table des matières

Liste des notations	ii
Liste des sigles et abréviations	iii
Table des figures	iv
Liste des tableaux	vii
1 Calcul numérique	1
Introduction générale	1
1.1 Préliminaires : Notion de fonction en Machine Learning	2
1.2 Dérivées d'une fonction d'une variable	2
1.3 Fonctions de plusieurs variables : Le monde réel de l'IA	4
1.4 Dérivées partielles de fonctions à plusieurs variables	4
1.5 Règle de la chaîne et différentiation composée	5
1.6 Gradient d'une fonction scalaire	6
1.7 Matrice jacobienne et matrice hessienne	8
1.8 Applications à l'optimisation	10
1.8.1 Points critiques et condition nécessaire d'optimalité	10
1.8.2 Convexité et caractérisation par la hessienne	11
1.8.3 Algorithme de descente de gradient	12
Points clés du chapitre	14
Exercices et Travaux Pratiques	15
2 Algèbre linéaire	17
Introduction générale	17
2.1 Scalaires, vecteurs, matrices, tenseurs et dimensionnalité	18
2.2 Produit matriciel, Trace, Rang et Compression Sémantique	19
2.3 Inversion Matricielle, Équations Normales et Conditionnement	21
2.4 Valeurs propres, Vecteurs propres et Dynamique des RNN	22
2.5 SVD : Décomposition en Valeurs Singulières et Espaces Latents	23
Points clés du chapitre	25
Exercices et Travaux Pratiques	26
3 Probabilités	28
Introduction générale	28
3.1 Espaces probabilisés, événements, axiomes de Kolmogorov	29
3.2 Probabilités conditionnelles, indépendance, théorème de Bayes	29
3.3 Variables aléatoires discrètes et continues	31
3.4 Masse, Densité et le Concept de Vraisemblance	32
3.5 Espérance, variance, moments : Le Risque Empirique	32
3.6 Lois usuelles : Le vocabulaire de la modélisation	33
Points clés du chapitre	35
Exercices et Travaux Pratiques	36

4	Statistiques	38
	Introduction générale	38
4.1	Statistiques descriptives : Tendances centrales, dispersion, quantiles	39
4.2	Graphiques et visualisations	41
4.3	Échantillonnage et estimation	42
4.4	Statistiques inférentielles : L'Évaluation Rigoureuse	44
4.5	Corrélation et notions de causalité : La Frontière de l'IA	46
	Points clés du chapitre	47
	Exercices et Travaux Pratiques	48
5	Mathématiques discrètes	49
	Introduction générale	49
5.1	Logique propositionnelle et des prédicats	50
5.2	Ensembles, relations, fonctions : Au cœur des Architectures	51
5.3	Combinatoire : Dénombrement et Explosion Dimensionnelle	52
5.4	Théorie des graphes : La structure sous-jacente de l'IA	53
5.5	Complexité Algorithmique : Le Prix de l'Intelligence	54
	Points clés du chapitre	57
	Exercices et Travaux Pratiques	58
6	Exercices d'application	60
6.1	Calcul Différentiel et Optimisation	60
6.2	Algèbre Linéaire et Espaces Latents	60
6.3	Probabilités, Inférence et Vraisemblance	61
6.4	Statistiques, EDA et Évaluation	62
6.5	Mathématiques Discrètes, Graphes et Complexité	62
6.6	Exercices Intégratifs de Synthèse (Multi-Chapitres)	63

Chapitre 1

Calcul numérique

Introduction générale

Le calcul différentiel constitue le socle analytique sur lequel repose la quasi-totalité des méthodes modernes d'apprentissage automatique (*Machine Learning*) et d'apprentissage profond (*Deep Learning*). Entraîner un modèle revient presque toujours à résoudre un problème d'optimisation : trouver les paramètres qui minimisent une fonction de coût mesurant l'écart entre les prédictions et les données réelles. Or, pour y parvenir, il faut savoir dans quelle direction et de combien cette fonction varie lorsqu'on modifie légèrement ses paramètres : c'est précisément le rôle du calcul différentiel.

Ce chapitre construit progressivement les outils nécessaires pour généraliser ces concepts aux fonctions à de très nombreuses variables, comme les poids d'un réseau de neurones. Après un rappel des règles de dérivation fondamentales, nous introduirons les trois objets centraux du chapitre : le **gradient**, qui indique la direction de plus forte variation ; la **matrice jacobienne**, qui le généralise aux fonctions vectorielles ; et la **matrice hessienne**, qui capture l'information de courbure (nécessaire pour identifier un minimum ou un point-selle).

La **règle de la chaîne** y joue un rôle particulier : appliquée systématiquement à la structure en couches d'un réseau de neurones, elle donne naissance à l'algorithme de *rétropropagation du gradient* (*backpropagation*). Le chapitre se conclut par l'application directe de ces outils avec la **descente de gradient**, brique algorithmique fondamentale de l'IA moderne.

Objectif

Objectif : maîtriser les outils du calcul différentiel nécessaires à l'optimisation des modèles de *Machine Learning*.

Masse horaire : 4 h de cours • 3 h de TD/TP.

• Outils & bibliothèques

Python, NumPy, SymPy

1.1 Préliminaires : Notion de fonction en Machine Learning

Avant d'aborder le calcul de dérivées, il est crucial de rappeler ce qu'est une fonction, car en Machine Learning, absolument tout est modélisé sous forme de fonctions.

Définition 1.1: Fonction

Une fonction f est une relation qui associe à chaque élément x d'un ensemble de départ E (le domaine) un unique élément y d'un ensemble d'arrivée F (le codomaine). On note :

$$f: E \rightarrow F, \quad x \mapsto f(x) = y$$

Interprétation en IA : Dans le contexte de l'Intelligence Artificielle, on rencontre principalement deux grands types de fonctions :

- **Le Modèle (Prédiction) :** Une fonction $f_\theta(x) = \hat{y}$ qui prend en entrée des données x (ex : les pixels d'une image) et retourne une prédiction $\hat{y}$ (ex : la probabilité que ce soit un chat). Cette fonction dépend de paramètres internes θ (les poids).
- **La Fonction de Coût (Optimisation) :** Une fonction $\mathcal{L}(\theta)$ qui évalue à quel point le modèle se trompe. Son ensemble de départ est l'espace des paramètres (souvent immense, $\mathbb{R}^n$), et son ensemble d'arrivée est un simple réel ($\mathbb{R}$). **L'objectif de l'apprentissage est de trouver le θ qui rend $\mathcal{L}(\theta)$ le plus petit possible.**

1.2 Dérivées d'une fonction d'une variable

Pour minimiser notre fonction de coût $\mathcal{L}$, nous devons comprendre comment elle varie. Commençons par le cas le plus simple : une fonction f qui ne dépend que d'une seule variable réelle x .

Définition 1.2: Dérivée en un point

Soit $f: I \rightarrow \mathbb{R}$ une fonction définie sur un intervalle ouvert $I \subset \mathbb{R}$, et soit $x_0 \in I$. On dit que f est **dérivable** en x_0 si la limite

$$f'(x_0) = \lim_{h \rightarrow 0} \frac{f(x_0 + h) - f(x_0)}{h}$$

existe et est finie. Le réel $f'(x_0)$ est alors appelé **nombre dérivé** de f en x_0 .

Remarque 1.1: Interprétation

Géométriquement, $f'(x_0)$ est la pente de la tangente à la courbe représentative de f au point $(x_0, f(x_0))$. En apprentissage automatique, $f'(x_0)$ mesure la *sensibilité* de f à une petite variation de son entrée au voisinage de x_0 — une notion centrale pour comprendre comment ajuster les paramètres d'un modèle. Si $f'(x_0)$ est grand (en valeur absolue), modifier légèrement x_0 modifiera fortement la sortie de f .

Propriété 1.1: Règles de dérivation

Soient $f, g: I \rightarrow \mathbb{R}$ deux fonctions dérivables en $x \in I$ et $\lambda \in \mathbb{R}$. Alors :

$$\begin{aligned}
 (\text{Linéarité}) \quad & (\lambda f + g)'(x) = \lambda f'(x) + g'(x), \\
 (\text{Produit}) \quad & (fg)'(x) = f'(x)g(x) + f(x)g'(x), \\
 (\text{Quotient}) \quad & \left(\frac{f}{g}\right)'(x) = \frac{f'(x)g(x) - f(x)g'(x)}{g(x)^2}, \quad g(x) \neq 0, \\
 (\text{Composée}) \quad & (g \circ f)'(x) = g'(f(x))f'(x).
 \end{aligned}$$

Fonction	Dérivée
$f(x) = x^n, n \in \mathbb{R}$	$f'(x) = n x^{n-1}$
$f(x) = e^x$	$f'(x) = e^x$
$f(x) = \ln(x)$	$f'(x) = \frac{1}{x}$
$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = \sigma(x)(1 - \sigma(x))$
$f(x) = \tanh(x)$	$f'(x) = 1 - \tanh^2(x)$

TABLE 1.1 – Dérivées usuelles, dont deux fonctions d'activation classiques en apprentissage profond (sigmoïde, tangente hyperbolique).

Exemple 1.1: Dérivée de la fonction sigmoïde

La fonction sigmoïde $\sigma(x) = (1 + e^{-x})^{-1}$ s'écrit comme composée $g \circ f$ avec $f(x) = 1 + e^{-x}$ et $g(u) = u^{-1}$. La règle de la fonction composée donne

$$\sigma'(x) = -(1 + e^{-x})^{-2} \cdot (-e^{-x}) = \frac{e^{-x}}{(1 + e^{-x})^2} = \sigma(x)(1 - \sigma(x)),$$

en utilisant $1 - \sigma(x) = \frac{e^{-x}}{1 + e^{-x}}$. Cette identité est ce qui rend la sigmoïde particulièrement économique à différencier lors de la rétropropagation : on n'a pas besoin de recalculer des exponentielles, on réutilise simplement la valeur de la fonction elle-même !

Application :

Soit la fonction de perte locale typique pour un neurone linéaire avec cible $y = 3$: $\mathcal{L}(w) = \frac{1}{2}(2w - 3)^2$. Calculez $\mathcal{L}'(w)$.

Correction :

Ici, $\mathcal{L}$ est une fonction composée. Posons $u(w) = 2w - 3$. Alors $\mathcal{L}(w) = \frac{1}{2}u(w)^2$.

D'après la règle de la chaîne pour une variable : $\mathcal{L}'(w) = \frac{1}{2} \times 2u(w) \times u'(w)$.

Comme $u'(w) = 2$, on a : $\mathcal{L}'(w) = (2w - 3) \times 2 = 4w - 6$.

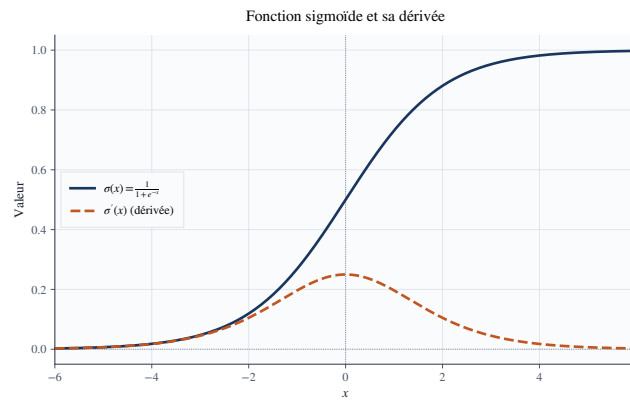


FIGURE 1.1 – La fonction sigmoïde et sa dérivée. On observe que la dérivée est maximale en $x = 0$ (valeur 0.25) et tend rapidement vers 0 pour $|x|$ grand. C'est la cause du problème d'évanouissement du gradient dans les réseaux profonds.

1.3 Fonctions de plusieurs variables : Le monde réel de l'IA

Dans la section précédente, nous avons étudié $f: \mathbb{R} \rightarrow \mathbb{R}$. Mais un modèle de Machine Learning possède rarement un seul paramètre. Un simple réseau de neurones peut en posséder des millions.

Avant de parler de dérivées partielles, il faut comprendre les **fonctions à plusieurs variables**. Ce sont des fonctions du type :

$$f: \mathbb{R}^n \rightarrow \mathbb{R}, \quad \mathbf{x} = (x_1, x_2, \dots, x_n) \mapsto f(x_1, x_2, \dots, x_n)$$

L'entrée est un vecteur (une liste de nombres), et la sortie est un scalaire (un nombre unique, par exemple l'erreur globale du modèle).

Représentation et Lignes de niveau : Pour $n = 2$ (deux paramètres, par exemple le poids w et le biais b), on peut visualiser $f(w, b)$ comme une surface en 3D (une montagne). Les "lignes de niveau" (*contour plots*) relient les points (w, b) pour lesquels la fonction vaut la même chose (l'erreur est constante). L'optimisation consiste à trouver le fond de la vallée.

1.4 Dérivées partielles de fonctions à plusieurs variables

Puisque nous sommes sur une surface multidimensionnelle, la notion de "pente" dépend de la direction dans laquelle on regarde. C'est l'essence de la dérivée partielle.

Définition 1.3: Dérivée partielle

Soit $f: U \rightarrow \mathbb{R}$ définie sur un ouvert $U \subset \mathbb{R}^n$, et soit $\mathbf{x} = (x_1, \dots, x_n) \in U$. La **dérivée partielle** de f par rapport à x_i en $\mathbf{x}$ est

$$\frac{\partial f}{\partial x_i}(\mathbf{x}) = \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_i + h, \dots, x_n) - f(\mathbf{x})}{h},$$

lorsque cette limite existe. Elle mesure le taux de variation de f lorsque x_i varie, les autres coordonnées étant maintenues fixes.

Interprétation pratique : En IA, si f est notre fonction de coût, $\frac{\partial f}{\partial w_1}$ nous dit : "Si je gèle tous les poids de mon réseau sauf w_1 , de combien mon erreur va-t-elle monter ou descendre si je modifie un tout petit peu w_1 ?".

Exemple 1.2: Calcul de dérivées partielles

Soit $f(x, y) = x^2y + \sin(xy)$. Alors

$$\frac{\partial f}{\partial x}(x, y) = 2xy + y \cos(xy), \quad \frac{\partial f}{\partial y}(x, y) = x^2 + x \cos(xy).$$

Explication : Pour trouver $\frac{\partial f}{\partial x}$, on considère y comme une constante stricte (comme si c'était le nombre 5). La dérivée de $x^2 \times (\text{constante})$ est $2x \times (\text{constante})$. La dérivée de $\sin(x \times \text{constante})$ est $\text{constante} \times \cos(x \times \text{constante})$ (règle de la chaîne). D'où le résultat. On fait l'inverse pour $\frac{\partial f}{\partial y}$ en figeant x .

Exercice d'application (Erreur quadratique) :

Soit la fonction d'erreur d'un modèle de régression linéaire pour un point de donnée ($x = 2, y = 5$) : $\mathcal{L}(w, b) = (2w + b - 5)^2$. Calculez $\frac{\partial \mathcal{L}}{\partial w}$ et $\frac{\partial \mathcal{L}}{\partial b}$.

Correction :

Posons $u(w, b) = 2w + b - 5$. Ainsi $\mathcal{L}(w, b) = u(w, b)^2$.

1. Pour $\frac{\partial \mathcal{L}}{\partial w}$, b est constant. $\frac{\partial u}{\partial w} = 2$.

Donc $\frac{\partial \mathcal{L}}{\partial w} = 2 \times u(w, b) \times \frac{\partial u}{\partial w} = 2(2w + b - 5) \times 2 = 4(2w + b - 5)$.

2. Pour $\frac{\partial \mathcal{L}}{\partial b}$, w est constant. $\frac{\partial u}{\partial b} = 1$.

Donc $\frac{\partial \mathcal{L}}{\partial b} = 2 \times u(w, b) \times \frac{\partial u}{\partial b} = 2(2w + b - 5) \times 1 = 2(2w + b - 5)$.

Remarque 1.2: Vers le calcul symbolique

Sous SymPy, ces calculs se vérifient avec `sympy.diff(f, x)` et `sympy.diff(f, y)`. En pratique, pour les fonctions de coût utilisées en Machine Learning, ces calculs sont automatisés par *différentiation automatique* (Autograd dans PyTorch ou GradientTape dans TensorFlow) plutôt que par dérivation symbolique mais la compréhension du calcul manuel reste indispensable pour interpréter les résultats et déboguer les modèles.

1.5 Règle de la chaîne et différentiation composée

L'architecture d'un réseau de neurones (Deep Learning) est fondamentalement une composition géante de fonctions mathématiques. Une couche est imbriquée dans une autre : $f(x) = f_3(f_2(f_1(x)))$. Pour dériver cela, nous avons impérativement besoin de la version vectorielle de la règle de la chaîne.

Théorème 1.1: Règle de la chaîne (cas vectoriel)

Soit $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ différentiable en $\mathbf{x}$ et $g: \mathbb{R}^m \rightarrow \mathbb{R}^p$ différentiable en $f(\mathbf{x})$. Alors $h = g \circ f$ est différentiable en $\mathbf{x}$ et sa matrice jacobienne vérifie

$$J_h(\mathbf{x}) = J_g(f(\mathbf{x})) J_f(\mathbf{x}),$$

où le produit du membre de droite est un produit matriciel.

Démonstration. Par différentiabilité de g en $f(\mathbf{x})$ et de f en $\mathbf{x}$, on a les développements au premier ordre

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + J_f(\mathbf{x})\mathbf{h} + o(\|\mathbf{h}\|), \quad g(\mathbf{y} + \mathbf{k}) = g(\mathbf{y}) + J_g(\mathbf{y})\mathbf{k} + o(\|\mathbf{k}\|).$$

En posant $\mathbf{y} = f(\mathbf{x})$ et $\mathbf{k} = J_f(\mathbf{x})\mathbf{h} + o(\|\mathbf{h}\|)$, la composition donne

$$h(\mathbf{x} + \mathbf{h}) = g(\mathbf{y}) + J_g(\mathbf{y})J_f(\mathbf{x})\mathbf{h} + o(\|\mathbf{h}\|),$$

ce qui identifie $J_h(\mathbf{x}) = J_g(f(\mathbf{x}))J_f(\mathbf{x})$. ■

Remarque 1.3: Lien avec la rétropropagation

La règle de la chaîne est le principe mathématique fondateur de l'algorithme de **rétropropagation du gradient** : un réseau de neurones étant une composition de fonctions (couches), le gradient de la fonction de coût par rapport aux paramètres de chaque couche s'obtient en multipliant les jacobiniennes locales, de la sortie (l'erreur finale) vers l'entrée. C'est l'application du concept de *graphe de calcul*, où l'on fait circuler les dérivées à rebours via le produit matriciel stipulé par ce théorème.

Exemple 1.3: Composition scalaire

Soit $z = f(y)$ avec $y = g(x)$, $f(y) = y^2$ et $g(x) = \sin(x)$. Alors

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx} = 2y \cdot \cos(x) = 2\sin(x) \cos(x) = \sin(2x).$$

Ici, on voit l'élégance de la notation de Leibniz : "les dy s'annulent virtuellement".

1.6 Gradient d'une fonction scalaire

Regroupons maintenant nos dérivées partielles dans un objet mathématique unique et surpuissant.

Définition 1.4: Gradient

Soit $f: U \rightarrow \mathbb{R}$ différentiable sur un ouvert $U \subset \mathbb{R}^n$. Le **gradient** de f en $\mathbf{x} \in U$ est le vecteur des dérivées partielles

$$\nabla f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}) \\ \vdots \\ \frac{\partial f}{\partial x_n}(\mathbf{x}) \end{pmatrix} \in \mathbb{R}^n.$$

Propriété 1.2: Interprétation géométrique du gradient

En tout point $\mathbf{x}$ où $\nabla f(\mathbf{x}) \neq \mathbf{0}$:

1. $\nabla f(\mathbf{x})$ pointe dans la direction de plus forte croissance de f ; $-\nabla f(\mathbf{x})$ pointe dans la direction de plus forte *décroissance*;
2. $\nabla f(\mathbf{x})$ est orthogonal à la ligne (ou surface) de niveau de f passant par $\mathbf{x}$;
3. pour toute direction unitaire $\mathbf{u} \in \mathbb{R}^n$, la dérivée directionnelle de f en $\mathbf{x}$ dans la direction $\mathbf{u}$ vaut $\nabla f(\mathbf{x}) \cdot \mathbf{u}$, maximale lorsque $\mathbf{u}$ est colinéaire et de même sens que $\nabla f(\mathbf{x})$.

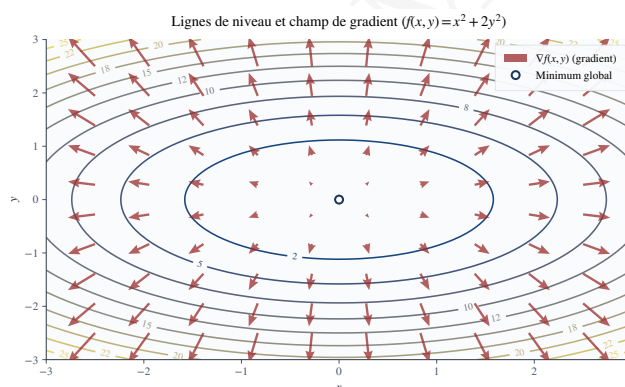


FIGURE 1.2 – Lignes de niveau d'une fonction de coût convexe quadratique. Les vecteurs rouges représentent le gradient ∇f . On remarque géométriquement qu'ils sont toujours orthogonaux aux lignes de niveau et pointent vers l'extérieur (direction de plus forte croissance).

Remarque 1.4: Pourquoi la descente de gradient fonctionne

C'est précisément le point (1) qui justifie l'algorithme de descente de gradient. Si vous êtes un aveugle cherchant le bas d'une vallée (minimiser l'erreur), la meilleure tactique locale est de tâter le sol autour de vous et de faire un pas dans la direction qui descend le plus vite : c'est la direction de $-\nabla f(\mathbf{x})$.

Exemple :

Soit $f(x, y) = x^2 + y^2$ (un bol parabolique dont le minimum est en $(0, 0)$). Calculons le gradient au point $A = (2, 1)$.

Correction :

Les dérivées partielles sont $\frac{\partial f}{\partial x} = 2x$ et $\frac{\partial f}{\partial y} = 2y$.

Donc $\nabla f(x, y) = \begin{pmatrix} 2x \\ 2y \end{pmatrix}$. Au point $A(2, 1)$, le gradient est $\nabla f(2, 1) = \begin{pmatrix} 4 \\ 2 \end{pmatrix}$.

Cela signifie que pour faire monter f le plus vite possible depuis le point $(2, 1)$, il faut avancer de 4 unités selon x pour chaque 2 unités selon y . Pour **minimiser** f (ce que l'on veut en ML), on fera un pas dans la direction *opposée* : $-\begin{pmatrix} 4 \\ 2 \end{pmatrix}$, ce qui pointe bel et bien vers l'origine $(0, 0)$!

1.7 Matrice jacobienne et matrice hessienne

Si le gradient est la "dérivée" d'une fonction scalaire, comment généraliser cela à une fonction vectorielle (qui sort un vecteur, comme une couche réseau de neurones)? Et comment généraliser la dérivée seconde?

Définition 1.5: Matrice jacobienne

Soit $f = (f_1, \dots, f_m): U \rightarrow \mathbb{R}^m$ différentiable sur un ouvert $U \subset \mathbb{R}^n$. La **matrice jacobienne** de f en $\mathbf{x} \in U$ est la matrice $m \times n$

$$J_f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1}(\mathbf{x}) & \cdots & \frac{\partial f_1}{\partial x_n}(\mathbf{x}) \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1}(\mathbf{x}) & \cdots & \frac{\partial f_m}{\partial x_n}(\mathbf{x}) \end{pmatrix}.$$

Lorsque $m = 1$ (la fonction est scalaire), la Jacobienne est simplement le gradient transposé : $J_f(\mathbf{x}) = \nabla f(\mathbf{x})^\top$.

Définition 1.6: Matrice hessienne

Soit $f: U \rightarrow \mathbb{R}$ deux fois différentiable sur un ouvert $U \subset \mathbb{R}^n$. La **matrice hessienne** de f en $\mathbf{x} \in U$ est la matrice symétrique $n \times n$ des dérivées partielles secondes

$$\nabla^2 f(\mathbf{x}) = \left(\frac{\partial^2 f}{\partial x_i \partial x_j}(\mathbf{x}) \right)_{1 \leq i, j \leq n},$$

la symétrie découlant du théorème de Schwarz sous hypothèse de continuité des dérivées secondes.

Propriété 1.3: Développement de Taylor à l'ordre deux

Soit $f: U \rightarrow \mathbb{R}$ de classe C^2 au voisinage de $\mathbf{x} \in U$. Alors, pour $\mathbf{h} \in \mathbb{R}^n$ proche de $\mathbf{0}$,

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \mathbf{h} + \frac{1}{2} \mathbf{h}^\top \nabla^2 f(\mathbf{x}) \mathbf{h} + o(\|\mathbf{h}\|^2).$$

Interprétation en Deep Learning : La formule de Taylor ci-dessus montre que la matrice Hessienne capture la **courbure** locale de la fonction de coût. Le gradient indique la pente, la Hessienne indique si cette pente s'aplatit (cuvette) ou s'accroît. Dans des espaces à millions de dimensions, la Hessienne aide à comprendre si l'on est dans un "minima plat" (bon pour la généralisation du modèle) ou un "minima aigu".

Exemple 1.4: Jacobienne et hessienne d'une fonction quadratique

Soit $f(\mathbf{x}) = \mathbf{x}^\top A \mathbf{x} + \mathbf{b}^\top \mathbf{x} + c$, avec $A \in \mathbb{R}^{n \times n}$ symétrique ($A^\top = A$), $\mathbf{b} \in \mathbb{R}^n$ et $c \in \mathbb{R}$. On obtient :

$$\nabla f(\mathbf{x}) = 2A\mathbf{x} + \mathbf{b}, \quad \nabla^2 f(\mathbf{x}) = 2A.$$

Démonstration :

Pour comprendre d'où viennent ces résultats, il faut réécrire $f(\mathbf{x})$ à l'aide de sommes scalaires (indices i et j). Soit x_i la i -ème composante du vecteur $\mathbf{x}$.

- Le terme constant c est un scalaire isolé.
- Le terme linéaire s'écrit : $\mathbf{b}^\top \mathbf{x} = \sum_{i=1}^n b_i x_i$.
- Le terme quadratique s'écrit : $\mathbf{x}^\top A \mathbf{x} = \sum_{i=1}^n \sum_{j=1}^n A_{i,j} x_i x_j$.

1. Calcul du Gradient $\nabla f(\mathbf{x})$

Calculons la dérivée partielle de f par rapport à une variable spécifique x_k (pour $k \in \{1, \dots, n\}$) :

$$\begin{aligned} \frac{\partial f}{\partial x_k} &= \frac{\partial}{\partial x_k} \left(\sum_{i=1}^n \sum_{j=1}^n A_{i,j} x_i x_j + \sum_{i=1}^n b_i x_i + c \right) \\ &= \sum_{i=1}^n \sum_{j=1}^n A_{i,j} \frac{\partial (x_i x_j)}{\partial x_k} + \sum_{i=1}^n b_i \frac{\partial x_i}{\partial x_k} + 0 \end{aligned}$$

Or, $\frac{\partial x_i}{\partial x_k}$ vaut 1 si $i = k$, et 0 sinon (symbole de Kronecker). Pour le terme linéaire :

$$\sum_{i=1}^n b_i \frac{\partial x_i}{\partial x_k} = b_k$$

Pour le terme quadratique, on applique la règle du produit $\frac{\partial (uv)}{\partial x} = u'v + uv'$:

$$\frac{\partial (x_i x_j)}{\partial x_k} = \left(\frac{\partial x_i}{\partial x_k} \right) x_j + x_i \left(\frac{\partial x_j}{\partial x_k} \right)$$

En injectant cela dans la double somme, on obtient deux termes :

$$\sum_{i=1}^n \sum_{j=1}^n A_{i,j} \left(\frac{\partial x_i}{\partial x_k} x_j + x_i \frac{\partial x_j}{\partial x_k} \right) = \sum_{j=1}^n A_{k,j} x_j + \sum_{i=1}^n A_{i,k} x_i$$

Le premier terme est la k -ème composante du vecteur $A\mathbf{x}$.

Le second terme est la k -ème composante du vecteur $A^\top \mathbf{x}$.

Puisque la matrice A est symétrique ($A = A^\top$), on a $A_{i,k} = A_{k,i}$. Donc les deux sommes sont identiques et valent $2 \sum_{j=1}^n A_{k,j} x_j$. En rassemblant le tout pour la composante k :

$$\frac{\partial f}{\partial x_k} = 2(A\mathbf{x})_k + b_k$$

En repassant sous forme vectorielle, on retrouve bien : $\nabla f(\mathbf{x}) = 2A\mathbf{x} + \mathbf{b}$.

2. Calcul de la Hessienne $\nabla^2 f(\mathbf{x})$

La matrice hessienne est la matrice des dérivées secondes. Soit la fonction composante

du gradient $g_k(\mathbf{x}) = \frac{\partial f}{\partial x_k} = 2 \sum_{j=1}^n A_{k,j} x_j + b_k$.

Dérivons g_k par rapport à une variable x_l :

$$\begin{aligned} \frac{\partial^2 f}{\partial x_k \partial x_l} &= \frac{\partial g_k}{\partial x_l} = \frac{\partial}{\partial x_l} \left(2 \sum_{j=1}^n A_{k,j} x_j + b_k \right) \\ &= 2 \sum_{j=1}^n A_{k,j} \frac{\partial x_j}{\partial x_l} + 0 \\ &= 2A_{k,l} \end{aligned}$$

Puisque l'élément à la position (k, l) de la matrice hessienne est exactement $2A_{k,l}$, on a sous forme matricielle : $\nabla^2 f(\mathbf{x}) = 2A$.

Interprétation en ML : Cette propriété est omniprésente. Dans une régression linéaire par moindres carrés, la fonction de coût est exactement de cette forme. La hessienne étant constante et égale à $2A$ (qui s'avère définie positive dans ce contexte), on a la preuve absolue que la surface d'erreur est un bol parfait (convexe), garantissant le succès de la descente de gradient vers un minimum global unique.

1.8 Applications à l'optimisation

1.8.1 Points critiques et condition nécessaire d'optimalité

L'objectif ultime est de trouver le "fond" de notre fonction de coût. Un fond est plat. Autrement dit, sa pente (son gradient) est nulle.

Définition 1.7: Point critique

Soit $f: U \rightarrow \mathbb{R}$ différentiable sur un ouvert $U \subset \mathbb{R}^n$. Un point $\mathbf{x}^* \in U$ est un **point critique** de f si $\nabla f(\mathbf{x}^*) = \mathbf{0}$.

Théorème 1.2: Condition nécessaire du premier ordre

Si $f: U \rightarrow \mathbb{R}$ est différentiable sur l'ouvert U et admet un extremum local (minimum ou maximum) en $\mathbf{x}^* \in U$, alors $\mathbf{x}^*$ est un point critique de f , c'est-à-dire $\nabla f(\mathbf{x}^*) = \mathbf{0}$.

Remarque 1.5: Condition nécessaire mais non suffisante

Un point critique peut être un minimum local, un maximum local, ou un point-selle (comme la forme d'une selle de cheval : un minimum dans une direction, un maximum dans une autre). Distinguer ces cas requiert une information d'ordre deux : les valeurs propres de la matrice hessienne (méthodes du second ordre, méthode de Newton).

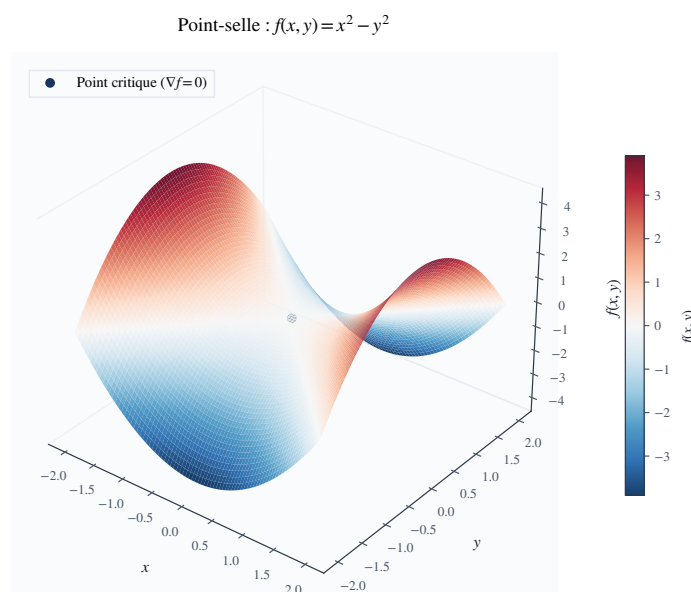


FIGURE 1.3 – Un point-selle (ou col) en $(0,0)$. Le gradient y est nul (le terrain est localement plat), mais ce n'est ni un minimum ni un maximum. L'analyse des valeurs propres de la matrice hessienne est indispensable pour identifier cette géométrie (ici, une valeur propre positive et une négative).

1.8.2 Convexité et caractérisation par la hessienne

En IA, nous aimons les fonctions *convexes*. Pourquoi ? Parce qu'un bol n'a qu'un seul fond. Tout point critique d'un bol est garanti d'être le minimum absolu.

Définition 1.8: Fonction convexe

Une fonction $f: \mathbb{R}^n \rightarrow \mathbb{R}$ est **convexe** si, pour tous $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ et tout $t \in [0, 1]$,

$$f(t\mathbf{x} + (1-t)\mathbf{y}) \leq tf(\mathbf{x}) + (1-t)f(\mathbf{y}).$$

Géométriquement, un segment reliant deux points de la courbe de f passe toujours au-dessus (ou sur) la courbe. Elle est **strictement convexe** si l'inégalité est stricte pour $\mathbf{x} \neq \mathbf{y}$ et $t \in (0, 1)$.

Théorème 1.3: Caractérisation de la convexité par la hessienne

Soit $f: \mathbb{R}^n \rightarrow \mathbb{R}$ de classe C^2 . Alors f est convexe si et seulement si $\nabla^2 f(\mathbf{x})$ est semi-définie positive pour tout $\mathbf{x} \in \mathbb{R}^n$. Si $\nabla^2 f(\mathbf{x})$ est définie positive pour tout $\mathbf{x}$, alors f est strictement convexe.

Propriété 1.4: Optimalité globale sous convexité

Si f est convexe et différentiable, alors tout point critique de f est un **minimum global**. Ce résultat justifie, en particulier, que pour une fonction de coût convexe (régression linéaire, régression logistique), un algorithme de descente de gradient convergeant vers un point critique converge vers l'optimum global. (Note : les réseaux de neurones profonds génèrent des fonctions de coût hautement **non-convexes**, d'où la complexité de leur optimisation).

1.8.3 Algorithme de descente de gradient

Voici enfin l'algorithme "roi" du Machine Learning moderne. C'est lui qui entraîne les modèles de génération de texte, la reconnaissance faciale, et les algorithmes de recommandation.

Algorithme 1 : Descente de gradient à pas constant

Entrées : fonction f différentiable (Coût), point initial $\mathbf{x}_0$ (Poids aléatoires initiaux), pas d'apprentissage (*learning rate*) $\alpha > 0$, tolérance $\varepsilon > 0$

Sorties : approximation $\mathbf{x}^*$ d'un point critique de f

```

1  $k \leftarrow 0$ ;
2 tant que  $\|\nabla f(\mathbf{x}_k)\| > \varepsilon$  faire
3    $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k)$ ; // Mise à jour dans le sens opposé au gradient
4    $k \leftarrow k + 1$ ;
5 fin
6 retourner  $\mathbf{x}_k$ 
```

Théorème 1.4: Convergence de la descente de gradient

Soit $f: \mathbb{R}^n \rightarrow \mathbb{R}$ différentiable, L -lisse (c'est-à-dire que ∇f est L -lipschitzienne) et bornée inférieurement. Si l'algorithme 1 est exécuté avec un pas constant $\alpha \in (0, 1/L]$, alors

$$\min_{0 \leq k \leq K-1} \|\nabla f(\mathbf{x}_k)\|^2 \leq \frac{2(f(\mathbf{x}_0) - f^*)}{\alpha K},$$

où $f^* = \inf_{\mathbf{x}} f(\mathbf{x})$. En particulier, le gradient tend vers 0 lorsque $K \rightarrow \infty$.

Démonstration. La L -lissité de f entraîne le lemme de descente

$$f(\mathbf{x}_{k+1}) \leq f(\mathbf{x}_k) - \alpha \|\nabla f(\mathbf{x}_k)\|^2 + \frac{L\alpha^2}{2} \|\nabla f(\mathbf{x}_k)\|^2.$$

Pour $\alpha \leq 1/L$, le coefficient de $\|\nabla f(\mathbf{x}_k)\|^2$ dans le membre de droite est majoré par $-\alpha/2$, d'où

$$f(\mathbf{x}_{k+1}) \leq f(\mathbf{x}_k) - \frac{\alpha}{2} \|\nabla f(\mathbf{x}_k)\|^2.$$

En sommant pour $k = 0, \dots, K - 1$ et en utilisant $f(\mathbf{x}_K) \geq f^*$, on obtient

$$\frac{\alpha}{2} \sum_{k=0}^{K-1} \|\nabla f(\mathbf{x}_k)\|^2 \leq f(\mathbf{x}_0) - f(\mathbf{x}_K) \leq f(\mathbf{x}_0) - f^*,$$

et il suffit de diviser par $\alpha K/2$ et de minorer la somme par K fois son plus petit terme. ■

Exemple 1.5: Descente de gradient sur une fonction quadratique

Pour $f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top A \mathbf{x}$ avec A symétrique définie positive de plus grande valeur propre L , on a $\nabla f(\mathbf{x}) = A \mathbf{x}$ et $\nabla^2 f(\mathbf{x}) = A$. Le choix du *learning rate* $\alpha = 1/L$ garantit la convergence de l'algorithme 1 vers l'unique minimum $\mathbf{x}^* = \mathbf{0}$, conformément au théorème 1.4.

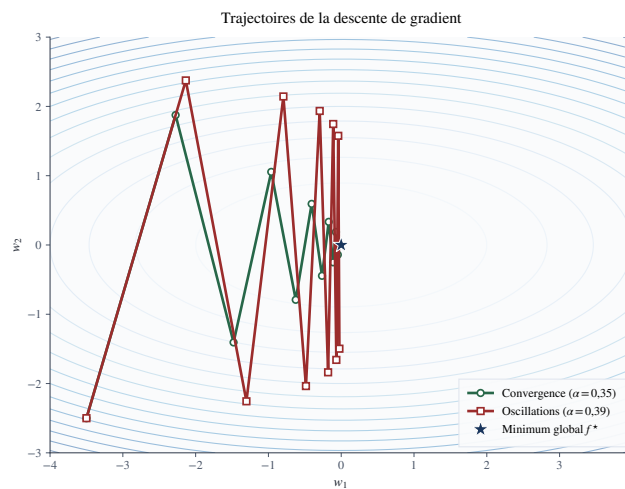


FIGURE 1.4 – Trajectoire des poids lors de la descente de gradient. Si le pas d'apprentissage α est bien calibré (vert), l'algorithme converge efficacement. S'il est trop grand (rouge), l'algorithme "rebondit" sur les parois de la vallée d'erreur (oscillations), ce qui ralentit considérablement la convergence, voire provoque une divergence.

Points clés du chapitre

Essentiels à retenir

- Un modèle d'IA et sa fonction de coût sont fondamentalement des **fonctions à plusieurs variables**.
- La dérivée partielle mesure la sensibilité locale d'une fonction par rapport à **un seul** paramètre, en figeant les autres.
- Le **gradient** rassemble ces dérivées partielles. Il indique la direction de plus forte croissance. **S'opposer** au gradient permet de minimiser l'erreur (Descente de gradient).
- La **jacobienne** généralise le gradient aux fonctions vectorielles; la **hessienne** encode l'information de courbure (dérivées secondes) et permet de diagnostiquer la nature d'un point critique (minimum, maximum, point-selle).
- La **règle de la chaîne** est le fondement mathématique de la **rétropropagation du gradient** (*backpropagation*), rendant possible l'entraînement de réseaux profonds.
- Un minimum local vérifie $\nabla f(\mathbf{x}^*) = \mathbf{0}$ (condition nécessaire); sous hypothèse de **convexité** de la fonction de coût, tout point critique est garanti d'être un minimum *global*.

Exercices et Travaux Pratiques (TP)

Partie I : Exercices Théoriques

1. Gradient de l'Erreur Quadratique (Régression Linéaire) :

En Machine Learning, l'erreur quadratique moyenne (MSE) pour un jeu de données de N points $\{(x_i, y_i)\}_{i=1}^N$ s'écrit pour un modèle linéaire $f(x) = wx + b$:

$$\mathcal{L}(w, b) = \frac{1}{2N} \sum_{i=1}^N (wx_i + b - y_i)^2$$

(a) Calculer les dérivées partielles $\frac{\partial \mathcal{L}}{\partial w}$ et $\frac{\partial \mathcal{L}}{\partial b}$.

(b) Montrer que si l'on pose $\nabla \mathcal{L}(w^*, b^*) = \mathbf{0}$, on retrouve les formules statistiques classiques de la covariance et de la variance pour w^* .

2. Calcul complet et optimisation analytique :

Soit la fonction $f(x, y) = x^2 e^y - 3xy^2$.

(a) Calculer le gradient $\nabla f(x, y)$.

(b) Calculer la matrice hessienne $\nabla^2 f(x, y)$.

(c) Évaluer le gradient et la hessienne au point $A = (1, 0)$. Dans quelle direction la fonction décroît-elle le plus vite depuis A ?

3. Distance, Gradient et Convexité :

Soit $g(\mathbf{x}) = \|\mathbf{x} - \mathbf{a}\|^2$ pour $\mathbf{a} \in \mathbb{R}^n$ fixé.

(a) Montrer, en développant la norme ou en utilisant la définition du gradient matriciel, que $\nabla g(\mathbf{x}) = 2(\mathbf{x} - \mathbf{a})$.

(b) En déduire l'unique point critique de g .

(c) Calculer la hessienne de g . Que peut-on en conclure sur la nature du point critique (minimum, maximum ou point-selle)?

4. Les limites du critère de la hessienne :

Montrer que la fonction $h(x) = x^4$ est strictement convexe sur $\mathbb{R}$ mais que sa dérivée seconde (hessienne en 1D) s'annule en $x = 0$. Que peut-on en conclure sur la réciproque du théorème 1.3?

Partie II : Travaux Pratiques sous Python (NumPy, SymPy, Matplotlib)

Ce TP a pour objectif de coder "from scratch" (sans bibliothèques de Machine Learning comme Scikit-Learn ou PyTorch) l'algorithme d'entraînement fondamental de l'IA : la Descente de Gradient.

5. Vérification symbolique avec SymPy :

Dans un *Notebook Jupyter*, importez `sympy`.

(a) Déclarez vos variables symboliques `x, y = sp.symbols('x y')`.

(b) Implémentez la fonction de l'exercice 2 : $f(x, y) = x^2 e^y - 3xy^2$.

(c) Utilisez `sp.diff()` pour calculer le gradient et `sp.hessian()` pour calculer la matrice hessienne. Comparez les résultats affichés avec vos calculs manuels.

6. Génération de données "Jouets" avec NumPy :

Simulons un jeu de données réel.

- (a) Générez un vecteur X de 100 valeurs aléatoires uniformes entre 0 et 10.
- (b) Générez un vecteur cible y tel que $y = 3x + 2 + \varepsilon$, où ε est un bruit gaussien $\mathcal{N}(0, 1)$ généré via `np.random.randn`.
- (c) Tracez le nuage de points avec `plt.scatter`.

7. Implémentation de la Descente de Gradient :

À partir de l'Algorithme 1 du cours :

- (a) Créez une fonction `compute_loss(w, b, X, y)` qui retourne la MSE.
- (b) Créez une fonction `compute_gradients(w, b, X, y)` qui retourne un tuple (dw, db) en utilisant les formules mathématiques trouvées à l'exercice 1.
- (c) Codez la boucle d'entraînement `gradient_descent(X, y, w_init, b_init, alpha, epochs)` qui met à jour w et b . Stockez la valeur de la perte (*loss*) à chaque itération dans une liste.

8. Visualisation de l'apprentissage (Courbe de perte) :

Exécutez votre algorithme avec $w_{init} = 0$, $b_{init} = 0$, un *learning rate* $\alpha = 0.01$ et 100 époques.

- (a) Tracez l'évolution de la fonction de perte en fonction du numéro de l'époque. Que remarquez-vous sur la forme de la courbe ?
- (b) Affichez la droite de régression obtenue $y_{pred} = w^*X + b^*$ sur le même graphique que le nuage de points de la question 6.

9. Exploration de la surface de coût et divergence (Hyperparamètres) :

Le choix du pas α (taux d'apprentissage) dépend directement de la constante de Lipschitz (liée à la courbure/Hessienne).

- (a) À l'aide de `plt.contour` ou `plt.contourf`, tracez les lignes de niveau de la fonction de coût $\mathcal{L}(w, b)$ dans l'espace des paramètres (faites varier $w \in [0, 5]$ et $b \in [0, 4]$).
- (b) Superposez sur ce graphique la trajectoire de vos poids (w_k, b_k) à chaque itération. Observez comment l'algorithme "descend" dans la vallée perpendiculairement aux lignes de niveau.
- (c) **Crash test** : Relancez l'entraînement avec un pas d'apprentissage trop grand (ex : $\alpha = 0.5$). Tracez à nouveau la trajectoire. Que se passe-t-il ? Reliez cette observation mathématiquement au Théorème 1.4.

Chapitre 2

Algèbre linéaire

Introduction générale

Si le calcul différentiel (Chapitre 1) indique *comment* ajuster un modèle, l'algèbre linéaire fournit le *langage* dans lequel les paramètres et les données sont représentés. L'apprentissage profond (*Deep Learning*) n'est fondamentalement qu'une succession de transformations géométriques fluides dans des espaces à très haute dimension. Pour l'ordinateur, l'image est une matrice et le texte un tenseur ; ignorer leur structure interne (rang, conditionnement, spectre) mène inévitablement à des modèles instables (les fameux *NaN* lors de l'entraînement) ou inefficaces en mémoire.

Ce chapitre pose rigoureusement ces fondations matricielles. Nous y définirons les **tenseurs** (structures reines de l'IA) et le *broadcasting*, avant de décortiquer le produit matriciel comme une composition géométrique. La notion de **rang** y sera directement reliée à la capacité de mémorisation d'un modèle (expliquant l'approche moderne *LoRA*).

Enfin, après avoir résolu la régression linéaire via les équations normales, nous insisterons sur l'impact vital du **conditionnement matriciel** sur l'optimisation. L'étude des **valeurs et vecteurs propres** (qui expliquent l'explosion des gradients dans les RNN) nous mènera à la Décomposition en Valeurs Singulières (**SVD**) : bien plus qu'un théorème élégant, c'est l'algorithme originel d'extraction de ce que l'IA appelle un « Espace Latent ».

Objectif

Objectif : Maîtriser les fondations géométriques et matricielles avancées du Machine Learning pour concevoir des architectures stables, comprendre la réduction de dimension, et penser en termes de *tenseurs* plutôt que de *boucles itératives*.

Masse horaire : 4 h de cours • 3 h de TD/TP.

• Outils & bibliothèques

NumPy, SciPy, Matplotlib

2.1 Scalaires, vecteurs, matrices, tenseurs et dimensionnalité

Dans les langages de programmation classiques, on traite les données séquentiellement via des boucles `for` ou `while`. En Intelligence Artificielle, cette approche est proscrite car elle ne permet pas d'exploiter les milliers de cœurs de calcul parallèles d'une carte graphique (GPU). L'unité fondamentale de calcul n'est donc pas le nombre isolé, mais le **tenseur**.

Définition 2.1: scalaire, vecteur, matrice, tenseur

En algèbre multilinéaire, on classifie les objets selon leur nombre d'axes (ou dimensions, ou *modes*) :

- **Scalaire (Ordre 0)** : Un simple nombre $a \in \mathbb{R}$. Il n'a aucun axe.
- **Vecteur (Ordre 1)** : Un tableau à un seul axe, $\mathbf{v} = (v_1, \dots, v_n)^\top \in \mathbb{R}^n$. Par convention stricte en algèbre, un vecteur est toujours un vecteur *colonne*.
- **Matrice (Ordre 2)** : Un tableau à deux axes (lignes et colonnes), $A \in \mathbb{R}^{m \times n}$, de coefficients A_{ij} (ligne i , colonne j).
- **Tenseur (Ordre $k \geq 3$)** : Un hyper-tableau multidimensionnel à k axes, $T \in \mathbb{R}^{n_1 \times n_2 \times \dots \times n_k}$.

Remarque 2.1: Anatomie d'un tenseur de données en Deep Learning

La forme (appelée *shape*) d'un tenseur dicte son organisation en mémoire. Prenons l'exemple d'un lot (*batch*) de 32 images couleurs en Haute Définition (1920×1080). Dans un framework comme PyTorch, ces données sont regroupées dans un tenseur d'ordre 4 de *shape* (32, 3, 1080, 1920) :

1. L'axe 0 est la **taille du batch** (32 images).
2. L'axe 1 est le nombre de **canaux de couleur** (3 pour Rouge, Vert, Bleu).
3. L'axe 2 est la **hauteur** spatiale (1080 pixels).
4. L'axe 3 est la **largeur** spatiale (1920 pixels).

Propriété 2.1: Broadcasting (Diffusion Numérique)

Le *broadcasting* est le mécanisme fondamental qui permet d'effectuer des opérations mathématiques entre des tenseurs de tailles différentes sans avoir à dupliquer explicitement les données en mémoire. Si l'on veut ajouter un vecteur de biais $\mathbf{b} \in \mathbb{R}^{1 \times n}$ à une matrice de données $X \in \mathbb{R}^{m \times n}$:

$$(X + \mathbf{b})_{ij} = X_{ij} + b_j$$

Le vecteur $\mathbf{b}$ est virtuellement "étiré" (diffusé) le long de l'axe des lignes (axe 0) pour correspondre à la taille m de la matrice X .

Bon à savoir : La convention de sommation d'Einstein (einsum)

Dans les architectures modernes (comme les Transformers à la base de ChatGPT), manipuler des tenseurs d'ordre 4 ou 5 devient un cauchemar syntaxique avec les produits matriciels classiques. L'opération `einsum` spécifie explicitement les axes à contracter via des chaînes de caractères.

- **Produit scalaire** $u^\top v : i, i \rightarrow$ (on somme sur l'axe i , le résultat n'a pas d'axe, c'est un scalaire).
- **Produit matriciel** $C = AB : ik, kj \rightarrow ij$ (on somme sur la dimension commune k , on garde i et j).
- **Mécanisme d'Attention (Multi-Head)** : Le calcul des scores d'attention requiert de multiplier un tenseur de Requêtes (Queries Q) et de Clés (Keys K).
Leur *shape* est $(\text{batch}, \text{heads}, \text{seq_len}, \text{d_model})$.
En `einsum`, cela s'écrit en une ligne de code sublime : `bhqd, bhkd -> bhqk`. On effectue un produit matriciel sur les dimensions de séquence (q, k) et de modèle (d), tout en appliquant cela indépendamment pour chaque exemple du batch (b) et chaque head (h).

2.2 Produit matriciel, Trace, Rang et Compression Sémantique

Définition 2.2: Produit matriciel comme Composition géométrique

Le produit matriciel $C = AB$ est défini analytiquement par $C_{ij} = \sum_k A_{ik}B_{kj}$. Cependant, géométriquement, si $B: \mathbb{R}^p \rightarrow \mathbb{R}^n$ et $A: \mathbb{R}^n \rightarrow \mathbb{R}^m$ sont des applications linéaires, alors la matrice AB représente l'**application composée** $A \circ B$, qui projette directement de $\mathbb{R}^p$ vers $\mathbb{R}^m$.

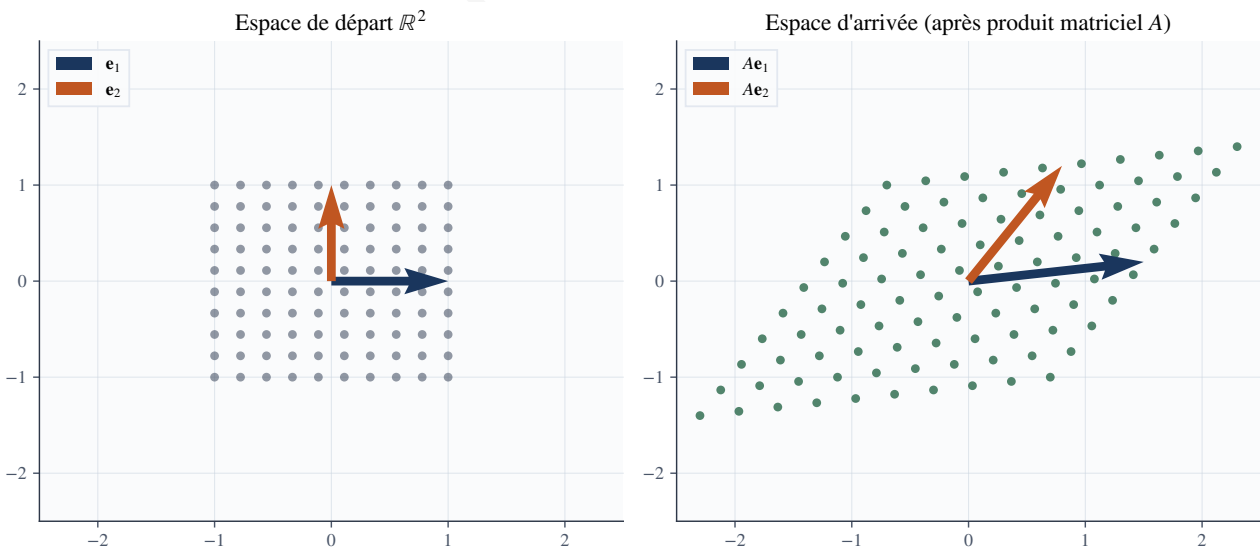


FIGURE 2.1 – Interprétation géométrique du produit matriciel. Une matrice $A \in \mathbb{R}^{2 \times 2}$ appliquée à une grille de points dans $\mathbb{R}^2$ agit comme une déformation de l'espace (ici une combinaison de rotation et d'étirement). L'aire d'un carré unitaire formé par les vecteurs de base devient un parallélogramme dont la surface est exactement égale au déterminant de A .

Exemple 2.1: Le calcul d'une couche de réseau de neurones en détail

Prenons une couche de réseau Dense (*Fully Connected*) recevant un lot de $N = 32$ observations, chacune contenant $d = 784$ caractéristiques (ex : pixels). L'entrée est la matrice $X \in \mathbb{R}^{32 \times 784}$. La couche contient $h = 128$ neurones. La matrice des poids est définie telle que chaque ligne représente un neurone : $W \in \mathbb{R}^{128 \times 784}$. Le vecteur de biais est $\mathbf{b} \in \mathbb{R}^{128}$. La transformation de toute la couche s'écrit en un seul produit matriciel :

$$Y = XW^T + \mathbf{b}$$

Analyse des dimensions :

1. W^T a pour dimension (784×128) .
2. Le produit $X \times W^T$ oppose une matrice (32×784) à une matrice (784×128) . L'axe intérieur (784) disparaît par sommation. Le résultat est une matrice (32×128) .
3. Le vecteur $\mathbf{b}$ de dimension (128) est ajouté à chaque ligne de la matrice résultante grâce au *broadcasting*.

Conclusion : Nous avons transformé 32 vecteurs de dimension 784 en 32 vecteurs de dimension 128 en une seule instruction optimisée pour le GPU.

Propriété 2.2: Transposée d'un produit

$(AB)^T = B^T A^T$. *Interprétation* : L'ordre s'inverse. En Deep Learning, lors de la passe avant (*forward*), l'information circule selon $x \mapsto Wx$. Lors de la passe arrière (*rétropropagation* du Chapitre 1), le gradient d'erreur δ remonte le graphe de calcul dans le sens opposé via la transposée : $\delta \mapsto W^T \delta$.

Définition 2.3: Rang et Indépendance Linéaire

Le **rang** d'une matrice $A \in \mathbb{R}^{m \times n}$, noté $\text{rg}(A)$, est le nombre maximal de colonnes (ou de lignes) qui sont **linéairement indépendantes**. Par exemple, si la colonne 3 est exactement égale à la somme de la colonne 1 et 2, elle n'apporte aucune information géométrique nouvelle. Le rang est la dimension réelle de l'espace généré (l'image de A). On a nécessairement $\text{rg}(A) \leq \min(m, n)$.

Interprétation ML : Effondrement Dimensionnel et LoRA

Considérez une matrice de poids d'un réseau immense $W \in \mathbb{R}^{10000 \times 10000}$. Elle contient 100 millions de paramètres. Si $\text{rg}(W) = 8$, cela signifie que malgré sa taille gigantesque, cette matrice écrase toute donnée d'entrée sur un sous-espace de seulement 8 dimensions ! Il y a énormément de redondance.

Cette observation est la base de **LoRA (Low-Rank Adaptation)**. Pour adapter un modèle LLM massif comme LLaMA sans réentraîner tous ses poids initiaux W_0 , on gèle W_0 et on apprend une petite perturbation ΔW . Au lieu d'apprendre 100 millions de valeurs, on force ΔW à être de rang faible en la décomposant : $\Delta W = AB$, où $A \in \mathbb{R}^{10000 \times 8}$ et $B \in \mathbb{R}^{8 \times 10000}$. Le nombre de paramètres à optimiser chute à $2 \times (10000 \times 8) = 160\,000$. **On vient de réduire la charge d'entraînement de 99.8 %**, rendant possible l'affinage de l'IA sur un simple ordinateur personnel.

2.3 Inversion Matricielle, Équations Normales et Conditionnement

Définition 2.4: Déterminant et Inverse

Le **déterminant** $\det(A)$ d'une matrice carrée mesure le facteur de dilatation du volume (ou aire) d'un hypercube après application de la transformation A . Si $\det(A) = 0$, le volume s'écrase totalement sur une dimension inférieure (le rang n'est pas plein). La matrice n'est pas **inversible** car on ne peut pas reconstruire un volume 3D à partir d'une ombre 2D (l'information perdue est irrécupérable).

Exemple 2.2: Régression Linéaire (Équations Normales)

L'algèbre linéaire permet de trouver le minimum absolu d'un modèle linéaire sans même utiliser la descente de gradient. Soit $X \in \mathbb{R}^{N \times d}$ la matrice de données et $y \in \mathbb{R}^N$ les cibles. La fonction de coût d'Erreur Quadratique (MSE) s'écrit sous forme vectorielle :

$$\mathcal{L}(w) = \|Xw - y\|_2^2 = (Xw - y)^\top (Xw - y)$$

Développons cette expression :

$$\mathcal{L}(w) = (w^\top X^\top - y^\top)(Xw - y) = w^\top X^\top Xw - w^\top X^\top y - y^\top Xw + y^\top y$$

Puisque le résultat est un scalaire, sa transposée est égale à lui-même : $w^\top X^\top y = (w^\top X^\top y)^\top = y^\top Xw$. On simplifie :

$$\mathcal{L}(w) = w^\top (X^\top X)w - 2w^\top X^\top y + y^\top y$$

Pour trouver le minimum, on calcule le gradient par rapport au vecteur w et on l'égalise au vecteur nul $\mathbf{0}$ (condition du 1^{er} ordre du Chapitre 1). En utilisant les identités de calcul matriciel $\nabla_w(w^\top Aw) = 2Aw$ (si A est symétrique) et $\nabla_w(w^\top b) = b$:

$$\nabla_w \mathcal{L}(w) = 2(X^\top X)w - 2X^\top y = \mathbf{0}$$

Ce qui nous donne directement les **Équations Normales** :

$$(X^\top X)w = X^\top y \implies w^* = (X^\top X)^{-1} X^\top y$$

Cependant, en Machine Learning, une matrice n'est presque jamais *exactement* non-inversible, mais elle peut être *presque* non-inversible. C'est ici qu'intervient le conditionnement.

Définition 2.5: Nombre de Conditionnement (κ)

Le **nombre de conditionnement** d'une matrice symétrique définie positive A (comme $X^\top X$ ou la Hessienne $\nabla^2 \mathcal{L}$) est le rapport entre sa plus grande et sa plus petite valeur propre :

$$\kappa(A) = \frac{\lambda_{\max}}{\lambda_{\min}}$$

Remarque 2.2: Le Ravin et l'Optimisation

Le conditionnement κ mesure la "rotondité" de la surface de coût. Si $\kappa \approx 1$ ($\lambda_{\max} \approx \lambda_{\min}$), la surface d'erreur est un bol parfait. La descente de gradient converge en ligne droite vers le centre en très peu d'itérations. Si $\kappa = 1000$ (une valeur propre est 1000 fois plus grande que l'autre), la surface est un ravin extrêmement allongé. Le gradient rebondira violemment sur les parois abruptes du ravin tout en avançant très lentement au fond. C'est la raison majeure pour laquelle on normalise toujours les données (StandardScaler) avant d'entraîner une IA : pour forcer le conditionnement de la matrice vers 1.

2.4 Valeurs propres, Vecteurs propres et Dynamique des RNN

Définition 2.6: Spectre d'une matrice

Un scalaire $\lambda \in \mathbb{C}$ est une **valeur propre** et $\mathbf{v} \neq \mathbf{0}$ son **vecteur propre** associé si la matrice A agit sur $\mathbf{v}$ de façon purement scalaire :

$$A\mathbf{v} = \lambda\mathbf{v}.$$

L'ensemble des valeurs propres est appelé le **spectre** de A . L'axe défini par $\mathbf{v}$ est invariant : la matrice ne fait qu'étirer ou contracter l'espace selon cet axe, avec une force λ .

Théorème 2.1: Théorème spectral (Le cœur de l'ACP)

Toute matrice **symétrique** $A \in \mathbb{R}^{n \times n}$ (comme une matrice de covariance) est diagonalisable dans une base orthonormée (ses vecteurs propres sont perpendiculaires entre eux). Il existe une matrice orthogonale Q ($Q^\top Q = I_n$) et une matrice diagonale Λ telles que :

$$A = Q\Lambda Q^\top$$

Toutes ses valeurs propres sont réelles. L'Analyse en Composantes Principales (ACP) utilise précisément Q pour projeter les données sur les axes de plus forte variance (plus fortes valeurs propres).

Exemple 2.3: Diagonalisation détaillée ligne par ligne

Prenons $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$. 1. **Valeurs propres** : Racines de $\det(A - \lambda I) = 0$.

$$\det \begin{pmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{pmatrix} = (2-\lambda)^2 - 1 = \lambda^2 - 4\lambda + 3 = 0.$$

Les solutions sont $\lambda_1 = 3$ et $\lambda_2 = 1$.

2. **Vecteurs propres** : Résoudre $A\mathbf{v} = \lambda\mathbf{v}$.

Pour $\lambda_1 = 3$: $\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = 3 \begin{pmatrix} x \\ y \end{pmatrix} \implies 2x + y = 3x \implies y = x$. Le vecteur est

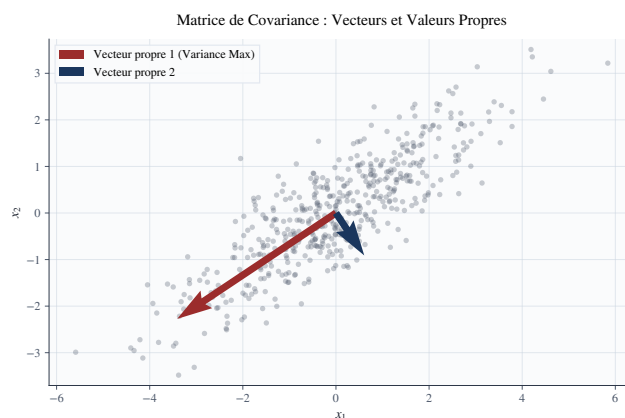


FIGURE 2.2 – Application du Théorème Spectral à une matrice de covariance. Le nuage de points génère une matrice symétrique définie positive. La diagonalisation révèle les axes naturels de la distribution : les vecteurs propres. La longueur de l’axe principal (en rouge) est proportionnelle à la racine de la valeur propre associée λ_1 , justifiant mathématiquement le principe de l’Analyse en Composantes Principales (ACP).

$$\mathbf{u}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Pour $\lambda_2 = 1 : 2x + y = x \implies y = -x$. Le vecteur est $\mathbf{u}_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$.

On remarque que $\mathbf{u}_1^\top \mathbf{u}_2 = \frac{1}{2}(1 - 1) = 0$. Ils sont bien orthogonaux !

Bon à savoir : Le Rayon Spectral et la mort des RNN

Dans un Réseau de Neurones Récurrent (RNN) utilisé pour les séries temporelles ou le texte, l’état caché h_t se met à jour selon une matrice de poids $W : h_t = Wh_{t-1}$. Déplier la récurrence sur T pas de temps donne $h_T = W^T h_0$.

Si on diagonalise $W = Q\Lambda Q^{-1}$, alors $W^T = Q\Lambda^T Q^{-1}$.

La limite quand $T \rightarrow \infty$ dépend de la plus grande valeur propre en valeur absolue, appelée **Rayon Spectral** $\rho(W)$:

- Si $\rho(W) > 1$, la valeur propre à la puissance T explose vers l’infini. C’est le problème de l’**Explosion du Gradient** (le réseau devient instable).
- Si $\rho(W) < 1$, $\lambda^T \rightarrow 0$. C’est le **Vanishing Gradient**. Le réseau oublie totalement le passé lointain (amnésie).

L’algèbre linéaire démontre ici pourquoi l’architecture RNN classique est défectueuse, justifiant la création des LSTMs ou des Transformers.

2.5 SVD : Décomposition en Valeurs Singulières et Espaces Latents

Le théorème spectral est magique, mais ne s’applique qu’aux matrices carrées symétriques. Que faire avec une matrice de données $X \in \mathbb{R}^{m \times n}$ quelconque (ex : Utilisateurs $\times$ Produits) ? La réponse est la SVD.

Théorème 2.2: Décomposition en valeurs singulières (SVD)

Toute matrice $X \in \mathbb{R}^{m \times n}$ (de rang r) admet une factorisation universelle :

$$X = U\Sigma V^\top$$

- $U \in \mathbb{R}^{m \times m}$ est une matrice orthogonale. Ses colonnes, les vecteurs singuliers gauches, décrivent la structure des lignes (ex : profils des utilisateurs).
- $V \in \mathbb{R}^{n \times n}$ est une matrice orthogonale. Ses colonnes, les vecteurs singuliers droits, décrivent la structure des colonnes (ex : caractéristiques des produits).
- $\Sigma \in \mathbb{R}^{m \times n}$ est une matrice diagonale contenant les **valeurs singulières** $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$. Elles représentent la "force" ou "l'importance" du signal capturé.

Théorème 2.3: Théorème d'Eckart-Young-Mirsky

Si on met à zéro toutes les valeurs singulières de Σ sauf les k premières, on obtient une matrice tronquée $\tilde{\Sigma}_k$. La matrice reconstituée :

$$X_k = U\tilde{\Sigma}_k V^\top = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$$

est la **meilleure approximation mathématique absolue** (Compression Optimale) de rang k de la matrice originale X (minimisant l'erreur quadratique $\|X - X_k\|_F^2$).

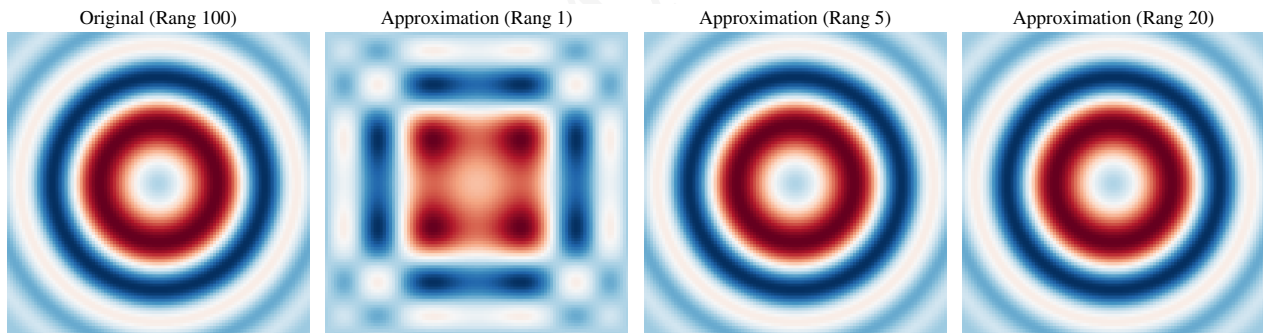


FIGURE 2.3 – Illustration du Théorème d'Eckart-Young via la Décomposition en Valeurs Singulières (SVD). Une matrice (ici représentée par une carte de chaleur de 100×100) de rang plein peut être approchée avec une fidélité étonnante en ne conservant qu'une fraction de ses valeurs singulières. Avec un rang $k = 20$, 99% de la variance structurelle est préservée pour un stockage drastiquement réduit.

L'essence de l'IA (Les Espaces Latents) :

Dans un système de recommandation de films (Netflix), X est une matrice immense, creuse et bruitée. En effectuant une SVD et en ne gardant que $k = 50$ composantes, on force les données à passer dans un **espace latent** restreint. La matrice ne décrit plus les films par leurs pixels ou leur titre, mais par 50 "concepts latents" cachés découverts par la machine (ex : le vecteur $\mathbf{u}_1$ pourrait représenter "l'affinité pour les films d'action des années 80"). C'est le fondement de la compréhension sémantique de l'IA.

Points clés du chapitre

Essentiels à retenir

- L'IA moderne requiert de **vectoriser** les opérations sous forme tensorielle pour saturer l'architecture des GPUs. Le *broadcasting* et *einsum* sont les outils syntaxiques de cette révolution.
- Le **rang** d'une matrice mesure la vraie quantité d'information linéairement indépendante qu'elle contient. Les matrices de rang faible (Low-Rank) permettent de compresser drastiquement les réseaux de neurones gigantesques.
- Le **conditionnement matriciel** régit la topologie du paysage d'erreur. Un mauvais conditionnement crée un "ravin" qui paralyse la descente de gradient classique.
- Le **rayon spectral** dicte la stabilité asymptotique des systèmes récurrents (RNN). Si $\rho \neq 1$, les gradients s'évanouissent ou explosent.
- La **SVD** ($X = U\Sigma V^T$) est la technique matricielle suprême. Couplée au théorème d'Eckart-Young, elle permet le débruitage, la compression, et surtout la projection des données dans des **espaces sémantiques latents**.

Exercices et Travaux Pratiques (TP)

Partie I : Exercices Théoriques (Calcul Matriciel & ML)

1. Inversion d'un bloc et stabilité :

Prouvez, en utilisant les propriétés des déterminants, que si la matrice des données X possède deux colonnes parfaitement colinéaires, alors le déterminant de la Hessienne de la régression linéaire ($X^T X$) est rigoureusement nul.

2. Semi-Définie Positivité et Convexité globale :

Soit la matrice de Gram $M = X^T X$.

- Montrez que pour tout vecteur $z \neq \mathbf{0}$, $z^T M z \geq 0$. (Indice : introduire le vecteur $v = Xz$ et utiliser la norme L_2).
- Sachant que la Hessienne de la fonction de coût de régression est proportionnelle à $X^T X$, que concluez-vous sur la topologie de la surface (Convexité) d'après le Chapitre 1 ?

3. Pseudo-Inverse de Moore-Penrose :

La SVD d'une matrice X rectangulaire est $X = U \Sigma V^T$. On définit la pseudo-inverse par $X^+ = V \Sigma^+ U^T$ (où Σ^+ s'obtient en inversant les éléments non nuls de Σ et en transposant la matrice). Prouvez la condition géométrique de Penrose : $XX^+X = X$.

Partie II : Travaux Pratiques sous Python (Du Ravin au Transformer)

Ce TP "from scratch" sous Python aborde les concepts les plus avancés de l'algèbre linéaire en IA, sans utiliser de frameworks de haut niveau.

4. Visualisation de la malédiction du Conditionnement (Le "Ravin") :

Considérez une fonction quadratique de coût $f(x, y) = \frac{1}{2}(ax^2 + by^2)$. Sa matrice Hessienne est diagonale avec pour valeurs propres a et b .

- Le Bol Parfait** : Fixez $a = 1, b = 1$ (Conditionnement $\kappa = 1$). Tracez les lignes de niveau (`plt.contour`). Implémentez une simple descente de gradient et superposez la trajectoire des poids. Observez la convergence rectiligne.
- Le Ravin** : Fixez $a = 1, b = 20$ (Conditionnement $\kappa = 20$). Tracez les lignes de niveau (des ellipses étirées). Relancez l'optimisation. Visualisez les oscillations destructrices (zig-zag). Expliquez pourquoi un grand rapport entre les valeurs propres ruine la dynamique d'apprentissage.

5. Implémentation de l'algorithme "Power Iteration" (SVD sous le capot) :

Derrière l'appel `np.linalg.svd`, se cache un algorithme itératif redoutable pour trouver la plus grande valeur singulière.

- Initialisez une matrice X aléatoire 50×30 et un vecteur unitaire aléatoire $\mathbf{v} \in \mathbb{R}^{30}$.
- Répétez 100 fois la boucle de projection suivante : projetez $\mathbf{u} = X\mathbf{v}$, puis mettez à jour $\mathbf{v} = X^T \mathbf{u}$. À chaque étape, normalisez $\mathbf{v} = \mathbf{v} / \|\mathbf{v}\|_2$.
- À la fin de la boucle, calculez $\sigma_1 = \|X\mathbf{v}\|_2$.
- Comparez ce scalaire avec la valeur renvoyée par `np.linalg.svd(X)[1][0]`. Conclusion ? Vous venez de coder la méthode mathématique à la base de l'algorithme de classement *PageRank* de Google.

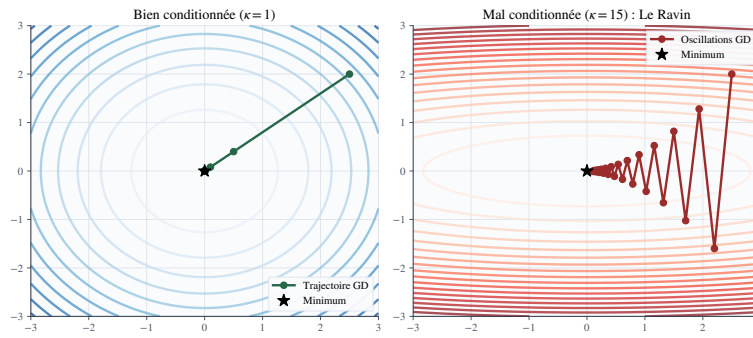


FIGURE 2.4 – Impact du conditionnement matriciel κ sur l'optimisation. À gauche : la matrice Hessienne est l'identité ($\kappa = 1$), les lignes de niveau sont parfaitement circulaires et l'algorithme converge en ligne droite. À droite : $\kappa = 15$ crée un "ravin". Le gradient (qui est orthogonal aux lignes de niveau elliptiques) pointe presque exclusivement vers les murs et très peu vers le minimum. L'algorithme rebondit violemment.

6. LE SUMMUM : Implémentation du mécanisme de Self-Attention :

L'architecture Transformer (Ex : ChatGPT) repose intégralement sur une seule équation d'algèbre linéaire :

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V$$

Soit une phrase de $N = 4$ mots, représentée par une matrice d'embeddings $X \in \mathbb{R}^{4 \times 3}$.

- Définissez X avec `np.random.randn`. Créez trois matrices de poids synaptiques arbitraires $W_Q, W_K, W_V \in \mathbb{R}^{3 \times 3}$.
- Calculez les projections linéaires : Les Requêtes ($Q = XW_Q$), Les Clés ($K = XW_K$), et Les Valeurs ($V = XW_V$).
- Calculez la matrice des scores bruts $S = QK^\top$. Quelle est sa dimension ? Que représente l'élément scalaire $S_{i,j}$ géométriquement entre le mot i et le mot j (pensez au produit scalaire) ?
- Divisez S par $\sqrt{3}$, puis appliquez la fonction Softmax par ligne (`scipy.special.softmax(S, axis=1)`). Vous obtenez la matrice d'attention A . Prouvez que la somme de chaque ligne vaut 1.
- Calculez enfin la sortie du module : $Z = AV$.

Analyse : Comparez la ligne 1 de Z avec la matrice V . Montrez comment l'algèbre linéaire permet au vecteur de sortie du mot 1 d'être une "combinaison linéaire convexe" des vecteurs de valeurs de *tous* les autres mots, créant ainsi le **contexte**.

Chapitre 3

Probabilités

Introduction générale

Un modèle de Machine Learning n'apprend jamais à partir de la totalité de la réalité qu'il cherche à décrire, mais à partir d'un échantillon fini, souvent bruité, de données. Prédire correctement à partir d'informations incomplètes n'est possible qu'à condition de raisonner en termes d'incertitude et le langage mathématique exclusif de l'incertitude est celui des probabilités.

L'Intelligence Artificielle moderne a cessé d'être purement déterministe. Qu'il s'agisse d'estimer la probabilité qu'un courriel soit indésirable (filtre anti-spam), de modéliser le bruit affectant le capteur d'un véhicule autonome, ou d'interpréter la sortie finale d'un réseau de neurones profond via une couche *Softmax*, la pensée probabiliste est omniprésente. Plus encore, l'ère de l'IA générative (ex : ChatGPT, Modèles de Diffusion, VAE) repose intégralement sur notre capacité à modéliser, échantillonner et manipuler des distributions de probabilités complexes.

Ce chapitre construit les fondations du raisonnement probabiliste orienté IA. On part des axiomes qui définissent un espace probabilisé, avant d'introduire les notions d'indépendance et de probabilité conditionnelle, couronnées par le **théorème de Bayes**, la pierre angulaire de l'inférence statistique et de familles entières de modèles. On formalise ensuite la notion de **variable aléatoire**, discrète ou continue, et les fonctions qui la caractérisent (masse, densité, répartition). On étudiera les résumés numériques (espérance, variance) en les reliant au concept d'apprentissage (minimisation du risque empirique), avant de passer en revue les **lois usuelles** (normale, binomiale, catégorielle) qui constituent le vocabulaire de base de toute fonction de perte (*Loss function*) en Deep Learning.

Objectif

Objectif : Maîtriser le raisonnement probabiliste pour comprendre comment l'IA quantifie l'incertitude, modélise le bruit et prend des décisions optimales.

Masse horaire : 3 h de cours • 2 h de TD/TP.

• Outils & bibliothèques

SciPy, Matplotlib, NumPy

3.1 Espaces probabilisés, événements, axiomes de Kolmogorov

Toute modélisation probabiliste commence par la définition de ce qui est "possible".

Définition 3.1: Univers, événement

On appelle **univers**, noté Ω , l'ensemble de toutes les issues possibles d'une expérience aléatoire. Un **événement** est une partie $A \subset \Omega$. L'ensemble des événements considérés, noté $\mathcal{F} \subset \mathcal{P}(\Omega)$, est une **tribu** : il contient Ω , est stable par complémentaire, et stable par réunion dénombrable.

Définition 3.2: Probabilité — axiomes de Kolmogorov

Une **probabilité** sur $(\Omega, \mathcal{F})$ est une application $\mathbb{P}: \mathcal{F} \rightarrow [0, 1]$ vérifiant :

(K1) $\mathbb{P}(\Omega) = 1$ (La certitude absolue que *quelque chose* va se produire).

(K2) pour toute suite $(A_n)_{n \geq 1}$ d'événements deux à deux disjoints (incompatibles),

$$\mathbb{P}\left(\bigcup_{n \geq 1} A_n\right) = \sum_{n \geq 1} \mathbb{P}(A_n) \quad (\sigma\text{-additivité}).$$

Le triplet $(\Omega, \mathcal{F}, \mathbb{P})$ est appelé **espace probabilisé**.

Interprétation ML : La couche Softmax et l'Axiome (K1)

En classification multi-classes (ex : reconnaître un chiffre entre 0 et 9), un réseau de neurones sort un vecteur de scores bruts $\mathbf{z} \in \mathbb{R}^{10}$ (les *logits*). Pour transformer ces scores abstraits en véritables **probabilités**, on utilise la fonction d'activation **Softmax** :

$$\mathbb{P}(\text{classe} = i \mid \mathbf{z}) = \frac{e^{z_i}}{\sum_{j=0}^9 e^{z_j}}$$

Cette fonction garantit mathématiquement que chaque probabilité est dans $[0, 1]$ et que la somme totale vaut exactement 1. Le réseau respecte ainsi nativement les axiomes de Kolmogorov !

3.2 Probabilités conditionnelles, indépendance, théorème de Bayes

L'IA consiste souvent à mettre à jour nos croyances à la lumière de nouvelles données. C'est l'essence des probabilités conditionnelles.

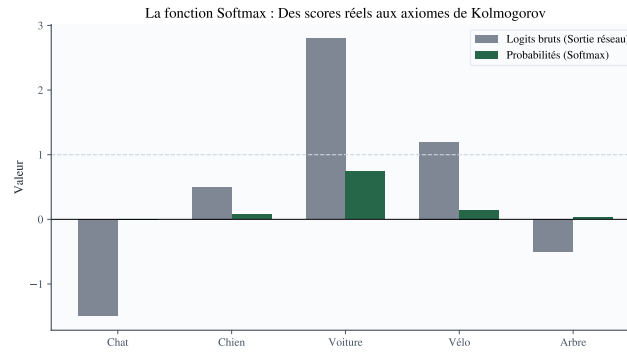


FIGURE 3.1 – Transformation des scores bruts (Logits) d'un réseau de neurones en probabilités via la fonction Softmax. On remarque que la Softmax garantit l'Axiome de Kolmogorov (K1) : toutes les valeurs sont positives et leur somme vaut exactement 1. De plus, elle amplifie les différences (la classe "Voiture" domine largement la distribution finale).

Définition 3.3: Probabilité conditionnelle

Soient $A, B \in \mathcal{F}$ avec $\mathbb{P}(B) > 0$. La **probabilité conditionnelle** de A sachant que B s'est réalisé est :

$$\mathbb{P}(A \mid B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}.$$

Elle représente la fraction de l'espace probabilisé de B qui est également occupée par A .

Définition 3.4: Indépendance de deux événements

Deux événements $A, B \in \mathcal{F}$ sont **indépendants** si $\mathbb{P}(A \cap B) = \mathbb{P}(A) \mathbb{P}(B)$. Lorsque $\mathbb{P}(B) > 0$, cela équivaut à $\mathbb{P}(A \mid B) = \mathbb{P}(A)$: **l'information apportée par B ne change absolument rien à notre prédiction sur A .**

Remarque 3.1: L'hypothèse I.I.D en Machine Learning

En Machine Learning, on suppose presque toujours que notre jeu de données de N exemples est constitué de variables **Indépendantes et Identiquement Distribuées (I.I.D)**. Cela permet d'écrire la probabilité d'observer tout le jeu de données comme le simple produit des probabilités individuelles : $\mathbb{P}(x_1, \dots, x_N) = \prod_{i=1}^N \mathbb{P}(x_i)$. Sans cette hypothèse, le calcul des fonctions de perte serait intraitable.

Théorème 3.1: Théorème de Bayes

Soit $(B_i)_{i \in I}$ un système complet d'événements et $A \in \mathcal{F}$ avec $\mathbb{P}(A) > 0$. Pour tout $j \in I$,

$$\mathbb{P}(B_j \mid A) = \frac{\mathbb{P}(A \mid B_j) \mathbb{P}(B_j)}{\sum_{i \in I} \mathbb{P}(A \mid B_i) \mathbb{P}(B_i)} = \frac{\mathbb{P}(A \mid B_j) \mathbb{P}(B_j)}{\mathbb{P}(A)}.$$

Bon à savoir : L'Inférence Bayésienne (Prior, Likelihood, Posterior)

Le Théorème de Bayes n'est pas qu'une formule, c'est une philosophie d'apprentissage. Remplaçons B_j par θ (les paramètres de notre modèle d'IA) et A par $\mathcal{D}$ (les données d'entraînement). La formule devient :

$$\mathbb{P}(\theta \mid \mathcal{D}) = \frac{\mathbb{P}(\mathcal{D} \mid \theta) \mathbb{P}(\theta)}{\mathbb{P}(\mathcal{D})}$$

- $\mathbb{P}(\theta)$: **L'a priori (Prior)**. Notre croyance sur les paramètres *avant* de voir les données.
- $\mathbb{P}(\mathcal{D} \mid \theta)$: **La Vraisemblance (Likelihood)**. La probabilité que ces paramètres précis génèrent les données observées.
- $\mathbb{P}(\theta \mid \mathcal{D})$: **L'a posteriori (Posterior)**. Notre croyance *mise à jour* après avoir vu les données.

Entraîner un modèle Bayésien, c'est maximiser cette probabilité *a posteriori** (MAP - Maximum A Posteriori). L'ajout d'une régularisation (L_2 /Ridge) en Machine Learning est mathématiquement strictement équivalent à imposer un **prior** Gaussien sur les poids θ !

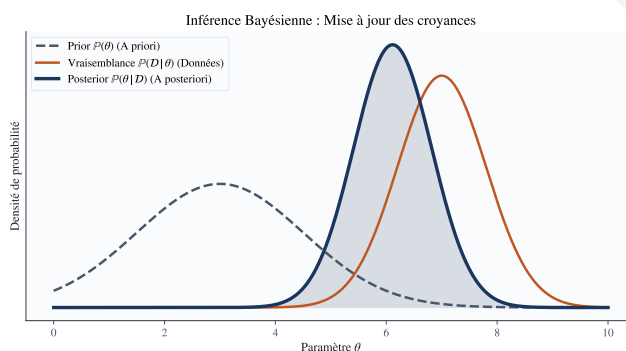


FIGURE 3.2 – L'Inférence Bayésienne en action. Avant de voir les données, notre croyance (Prior, en pointillés) est centrée sur 3. Les nouvelles observations (Vraisemblance, en orange) pointent fortement vers 7. Le Théorème de Bayes fusionne ces deux informations : la croyance mise à jour (Posterior, en bleu) est un compromis centré autour de 6. La variance du Posterior est plus faible, ce qui signifie que notre incertitude a diminué grâce à l'apprentissage.

3.3 Variables aléatoires discrètes et continues

Définition 3.5: Variable aléatoire

Une **variable aléatoire** sur $(\Omega, \mathcal{F}, \mathbb{P})$ est une application mesurable $X: \Omega \rightarrow \mathbb{R}$. Elle numérise les résultats d'une expérience. Elle est **discrète** si $X(\Omega)$ est fini ou dénombrable (ex : Catégories, mots d'un dictionnaire). Elle est **continue** (ou à densité) si elle prend des valeurs réelles non dénombrables (ex : Prix d'une maison, intensité d'un pixel).

3.4 Masse, Densité et le Concept de Vraisemblance

Définition 3.6: Fonction de masse (Discret)

Soit X une variable aléatoire discrète. Sa **fonction de masse** est $p_X(x) = \mathbb{P}(X = x)$. La somme sur toutes les valeurs vaut 1.

Définition 3.7: Fonction de densité (Continu)

Soit X une variable continue. Sa **densité de probabilité** $f_X: \mathbb{R} \rightarrow \mathbb{R}_+$ vérifie $\mathbb{P}(a \leq X \leq b) = \int_a^b f_X(x) dx$ et $\int_{-\infty}^{+\infty} f_X(x) dx = 1$. Contrairement au cas discret, $f_X(x)$ n'est *pas* une probabilité (elle peut dépasser 1); la probabilité d'un point précis est toujours nulle : $\mathbb{P}(X = x) = 0$. L'incertitude se mesure sur des intervalles.

Le cœur du Deep Learning : L'Entropie Croisée (Cross-Entropy Loss)

Comment un réseau de neurones apprend-il à classer des images? Par le principe de **Maximum de Vraisemblance**. Pour un exemple x_i de classe réelle y_i , le réseau prédit une fonction de masse $p_\theta(y | x_i)$. On veut maximiser la probabilité prédite de la bonne classe. Pour des raisons de stabilité numérique, au lieu de maximiser un produit de probabilités (qui tend vers 0 et cause de l'*underflow*), on minimise l'opposé du logarithme de la vraisemblance (NLL) :

$$\mathcal{L}(\theta) = - \sum_{i=1}^N \log(p_\theta(y_i | x_i))$$

Cette formule exacte porte le nom célèbre de **Cross-Entropy Loss**. C'est la fonction d'erreur utilisée par 99% des algorithmes de classification actuels !

3.5 Espérance, variance, moments : Le Risque Empirique

Définition 3.8: Espérance (La moyenne théorique)

L'**espérance** d'une variable aléatoire X est le "centre de gravité" de sa distribution :

$$\mathbb{E}[X] = \sum x_k p_X(x_k) \quad (\text{discret}), \quad \mathbb{E}[X] = \int x f_X(x) dx \quad (\text{continu}).$$

Remarque 3.2: Espérance et "Minimisation du Risque"

En apprentissage automatique, le but absolu est de minimiser l'erreur *espérée* sur des données futures invisibles. C'est le **Risque Vrai** : $R(\theta) = \mathbb{E}_{(x,y)}[\text{Loss}(f_\theta(x), y)]$. Ne connaissant pas la vraie distribution de la nature, on la remplace par une moyenne arithmétique sur nos données d'entraînement : le **Risque Empirique**. La Loi des Grands Nombres garantit que ce remplacement est valide si le dataset est assez grand.

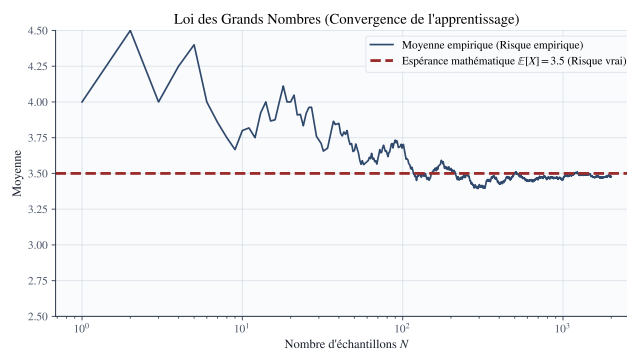


FIGURE 3.3 – Simulation de la Loi des Grands Nombres. Bien que les premiers lancers d'un dé soient soumis à un fort chaos aléatoire (variance élevée), la moyenne empirique converge inexorablement vers l'espérance mathématique théorique de 3.5 à mesure que la taille de l'échantillon N augmente. C'est ce théorème qui garantit que l'apprentissage sur un grand jeu de données (*Risque Empirique*) généralisera bien dans le monde réel (*Risque Vrai*).

Définition 3.9: Variance (La dispersion)

La **variance** mesure la dispersion autour de l'espérance (le Bruit) :

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - \mathbb{E}[X]^2$$

Propriété 3.1: Compromis Biais-Variance en IA

L'erreur d'un modèle d'IA se décompose mathématiquement en trois termes probabilistes : Erreur = Biais² + Variance + Bruit Irréductible. Un modèle trop simple (ex : régression linéaire) a un fort Biais (sous-apprentissage). Un modèle trop complexe (ex : réseau profond géant) a une forte Variance (sur-apprentissage, il mémorise par cœur le bruit). Trouver l'équilibre probabiliste est l'art de l'ingénieur IA.

3.6 Lois usuelles : Le vocabulaire de la modélisation

Définition 3.10: Loi de Bernoulli (Classification Binaire)

$X \sim \mathcal{B}(p)$: Modélise une réponse binaire (Vrai/Faux, Chien/Chat). $X = 1$ avec probabilité p et 0 avec probabilité $1 - p$.

Définition 3.11: Loi Catégorielle / Multinoulli

Généralisation de Bernoulli pour K classes distinctes (Classification Multiclasses). Un dé truqué à K faces. C'est la loi modélisée par la fonction **Softmax** des réseaux de neurones.

Définition 3.12: Loi Binomiale et Loi de Poisson

La **Binomiale** $\mathcal{B}(n, p)$ compte le nombre de succès sur n tirages de Bernoulli indépendants. La **Loi de Poisson** $\mathcal{P}(\lambda)$ (limite de la binomiale) modélise le nombre d'occurrences d'un événement rare (ex : nombre de clics sur une publicité, trafic

serveur) avec $\mathbb{E}[X] = \text{Var}(X) = \lambda$.

Définition 3.13: Loi Normale / Gaussienne (Le Bruit Universel)

$X \sim \mathcal{N}(\mu, \sigma^2)$ admet pour densité la célèbre courbe en cloche :

$$f_X(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Loi	Espérance	Variance
Bernoulli $\mathcal{B}(p)$	p	$p(1-p)$
Binomiale $\mathcal{B}(n, p)$	np	$np(1-p)$
Poisson $\mathcal{P}(\lambda)$	λ	λ
Uniforme continue $\mathcal{U}([a, b])$	$\frac{a+b}{2}$	$\frac{(b-a)^2}{12}$
Normale $\mathcal{N}(\mu, \sigma^2)$	μ	σ^2

TABLE 3.1 – Espérance et variance des lois usuelles.

Remarque 3.3: Omniprésence de la loi Gaussienne en IA

Grâce au **Théorème Central Limite (TCL)**, la somme d'un grand nombre de petits effets aléatoires indépendants tend vers une loi normale. C'est pourquoi on modélise presque toujours le bruit d'observation des capteurs par du bruit Gaussien. De plus, la loi normale est celle qui maximise l'**Entropie** (la forme la plus incertaine/aléatoire possible) pour une moyenne et variance données.

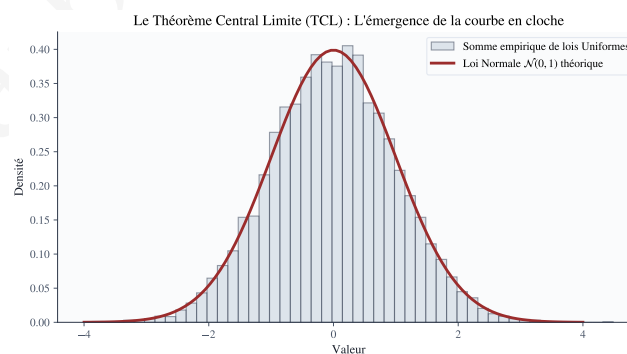


FIGURE 3.4 – Le Théorème Central Limite. L'histogramme empirique (en gris) montre le résultat de l'addition de seulement 12 variables aléatoires de loi Uniforme (qui n'ont pas du tout une forme de cloche). Magiquement, leur somme épouse presque parfaitement la courbe théorique de la loi Normale (en rouge). Ce phénomène justifie l'utilisation systématique de la loi Gaussienne pour modéliser le bruit des capteurs physiques ou initialiser les poids des réseaux de neurones profonds.

Points clés du chapitre

Essentiels à retenir

- Un espace probabilisé formalise le domaine des possibles. Les fonctions d'activation comme **Softmax** garantissent mathématiquement le respect des axiomes de Kolmogorov.
- Le **théorème de Bayes** est la théorie fondamentale de l'apprentissage : il explique comment mettre à jour nos *priors* en fonction de la *vraisemblance* des nouvelles données.
- L'entraînement d'un réseau de neurones par **Cross-Entropy** est strictement équivalent à la minimisation de la Log-Vraisemblance Négative d'une variable aléatoire.
- L'**espérance** définit le concept de Risque (Erreur moyenne à minimiser), tandis que la **variance** est au cœur des problématiques de sur-apprentissage (Overfitting).
- La loi **Normale (Gaussienne)** est la fondation continue universelle pour modéliser le bruit de régression, initialiser les poids des réseaux, ou structurer l'espace latent des modèles génératifs.

Exercices et Travaux Pratiques (TP)

Partie I : Exercices Théoriques (Inférence et Moments)

1. Le paradoxe du Test Médical (Théorème de Bayes) :

Une IA est conçue pour détecter une tumeur maligne sur des radiographies. La prévalence de la tumeur est $\mathbb{P}(T) = 0.005$ (0.5%). Le modèle IA a une sensibilité de 98% ($\mathbb{P}(\text{Positif} \mid T) = 0.98$) et un taux de faux positifs de 3% ($\mathbb{P}(\text{Positif} \mid T^c) = 0.03$).

- Calculez la probabilité totale que l'IA prédise un cas Positif.
- Si l'IA signale une tumeur, quelle est la probabilité réelle que le patient soit malade $\mathbb{P}(T \mid \text{Positif})$?
- Commentez l'impact du *Prior* très faible (0.5%) sur la confiance que l'on peut accorder au modèle, malgré ses métriques de performance apparentes de plus de 90%.

2. Propriétés de l'Espérance et de la Variance :

Soient $X_1, X_2, \dots, X_N$ des variables aléatoires **I.I.D** (Indépendantes et Identiquement Distribuées), chacune d'espérance μ et de variance σ^2 . Soit la moyenne empirique

$$\bar{X} = \frac{1}{N} \sum_{i=1}^N X_i.$$

- En utilisant la linéarité de l'espérance, prouvez que $\mathbb{E}[\bar{X}] = \mu$ (L'estimateur est *sans biais*).
- En utilisant les propriétés de la variance pour des variables indépendantes, montrez que $\text{Var}(\bar{X}) = \frac{\sigma^2}{N}$.
- Conclusion ML* : Que se passe-t-il pour la variance de notre estimation lorsque la taille du dataset N devient immense ?

3. Maximum de Vraisemblance (MLE) pour Bernoulli :

On lance une pièce truquée N fois. On obtient k Piles et $(N - k)$ Faces. Soit p la probabilité inconnue d'obtenir Pile. La fonction de vraisemblance est $L(p) = p^k(1 - p)^{N-k}$.

- Exprimez la Log-Vraisemblance $\ell(p) = \log(L(p))$.
- En calculant la dérivée $\frac{d\ell(p)}{dp}$ et en l'égalant à zéro, prouvez mathématiquement que la meilleure estimation possible pour p est la fréquence empirique $\hat{p} = k/N$.

Partie II : Travaux Pratiques sous Python (Simulation et TCL)

Ouvrez un *Notebook Jupyter*. L'objectif est d'utiliser NumPy et SciPy pour visualiser concrètement les grands théorèmes des probabilités.

4. Simulation de la Loi des Grands Nombres (LLN) :

La Loi des Grands Nombres stipule que la moyenne empirique d'un échantillon converge vers l'espérance mathématique théorique lorsque la taille de l'échantillon $N \rightarrow \infty$.

- Utilisez `np.random.randint(1, 7, size=N)` pour simuler $N = 10\,000$ lancers de dés à 6 faces (Uniforme discrète).

- (b) Calculez un vecteur contenant la moyenne cumulée pour chaque sous-échantillon de taille n (de 1 à 10 000) en utilisant `np.cumsum()`.
- (c) Tracez l'évolution de cette moyenne avec `Matplotlib`.
- (d) Tracez une ligne horizontale rouge représentant l'espérance théorique $\mathbb{E}[X] = 3.5$. Constatez visuellement la stabilisation du chaos aléatoire vers la vérité théorique.

5. **Démonstration visuelle du Théorème Central Limite (TCL) :**

Montrons que l'addition de phénomènes non-gaussiens génère une magnifique courbe de Gauss.

- (a) Générez une matrice $10\,000 \times 12$ remplie de valeurs issues d'une distribution uniforme $\mathcal{U}([0, 1])$ via `np.random.uniform`. (10000 expériences, chacune sommant 12 variables).
- (b) Calculez la somme des colonnes (axe 1) pour obtenir un vecteur de taille 10000.
- (c) Soustrayez 6 (qui est l'espérance de la somme de 12 uniformes) à ce vecteur pour le centrer.
- (d) Tracez l'histogramme de ce vecteur avec `bins=50` et `density=True`.
- (e) Superposez la courbe de densité théorique d'une loi Normale centrée réduite $\mathcal{N}(0, 1)$ en utilisant `scipy.stats.norm.pdf`. Que remarquez-vous sur la superposition ? L'hypothèse Gaussienne est-elle un mythe ou une loi de la nature ?

Chapitre 4

Statistiques

Introduction générale

Le chapitre précédent a posé le cadre probabiliste permettant de raisonner sur l'incertitude à partir d'un modèle supposé connu. La statistique renverse la perspective : partant de données observées, il s'agit d'en extraire de l'information, de les résumer, de les visualiser, et d'inférer les caractéristiques de la population ou du processus génératif dont elles sont issues. Toute la démarche du Machine Learning supervisé, de l'exploration initiale d'un jeu de données (*Exploratory Data Analysis*) à l'évaluation finale et rigoureuse d'un modèle en production, repose sur ces gestes statistiques élémentaires.

Ce chapitre s'ouvre sur les **statistiques descriptives**, tendances centrales, dispersion, quantiles qui condensent un jeu de données en quelques nombres interprétables, puis sur les principales représentations graphiques qui permettent d'en saisir la structure d'un coup d'œil. Nous verrons que ces simples mesures descriptives sont en réalité intimement liées aux fonctions de perte utilisées en *Deep Learning*. On introduit ensuite les notions d'**échantillonnage** et d'**estimation**, qui formalisent le passage d'un échantillon fini à des conclusions sur la population dont il est issu. Ces théorèmes fondamentaux justifient l'existence même de l'optimisation stochastique qui entraîne les réseaux de neurones.

Le cœur du chapitre porte sur les **statistiques inférentielles** : tests d'hypothèses, p -valeur, intervalles de confiance, les outils qui permettent de quantifier la confiance que l'on peut accorder à une conclusion tirée de données limitées. Sans ces outils, l'évaluation d'une IA relève de la conjecture. Le chapitre se conclut par la distinction, essentielle et trop souvent négligée dans l'IA actuelle, entre **corrélation** (ce que les modèles actuels exploitent) et **causalité** (ce qui permettrait à l'IA de véritablement comprendre le monde physique).

Objectif

Objectif : Savoir décrire, visualiser, inférer à partir de données, et évaluer rigoureusement la performance d'un modèle.

Masse horaire : 3h de Cours • 2h de TD/TP.

• Outils & bibliothèques

Pandas, Seaborn, Scipy.stats, Scikit-Learn

4.1 Statistiques descriptives : Tendances centrales, dispersion, quantiles

Définition 4.1: Moyenne empirique

Pour un échantillon $x_1, \dots, x_n \in \mathbb{R}$, la **moyenne empirique** est le centre de masse des données :

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.$$

Définition 4.2: Médiane

La **médiane** $\tilde{x}$ d'un échantillon $x_1, \dots, x_n$ est la valeur qui partage l'échantillon trié en deux moitiés égales : en notant $x_{(1)} \leq \dots \leq x_{(n)}$ l'échantillon ordonné,

$$\tilde{x} = \begin{cases} x_{(\frac{n+1}{2})} & \text{si } n \text{ est impair,} \\ \frac{1}{2} (x_{(\frac{n}{2})} + x_{(\frac{n}{2}+1)}) & \text{si } n \text{ est pair.} \end{cases}$$

Remarque 4.1: Sensibilité aux valeurs extrêmes

Contrairement à la moyenne, la médiane est **robuste** aux valeurs aberrantes (*outliers*) : modifier une seule observation, aussi extrême soit-elle, ne déplace la médiane que d'un cran au plus dans l'échantillon ordonné, alors que la moyenne peut être arbitrairement déplacée par une seule valeur corrompue.

Interprétation ML : Erreur Quadratique (L_2) vs Erreur Absolue (L_1)

L'algèbre prouve que la **moyenne** $\bar{x}$ est l'unique scalaire c qui minimise la somme des carrés des écarts : $\arg \min_c \sum_{i=1}^n (x_i - c)^2$. C'est pourquoi la fonction de perte **MSE** force un réseau de neurones à prédire l'espérance mathématique conditionnelle (ce qui le rend sensible aux *outliers*). À l'inverse, la **médiane** minimise l'erreur absolue : $\arg \min_c \sum_{i=1}^n |x_i - c|$. Utiliser la perte **MAE** force l'IA à prédire la médiane, garantissant une robustesse structurelle face aux données aberrantes.

Définition 4.3: Mode

Le **mode** d'un échantillon est la valeur (ou la classe de valeurs, pour des données groupées) la plus fréquemment observée. Il est particulièrement adapté aux variables catégorielles (classes de classification en ML), pour lesquelles moyenne et médiane ne sont pas définies.

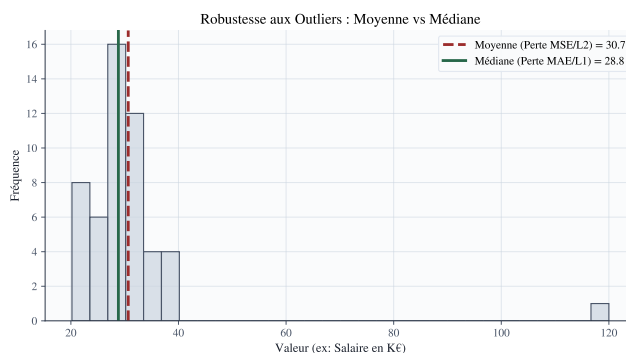


FIGURE 4.1 – Sensibilité aux anomalies (*outliers*). L’ajout d’une seule valeur aberrante extrême (ici 120) tire violemment la moyenne vers la droite. En revanche, la médiane reste parfaitement stable au centre de la distribution principale. C’est la raison géométrique pour laquelle la fonction de perte absolue (MAE, optimisant la médiane) est plus robuste que la perte quadratique (MSE, optimisant la moyenne) en *Machine Learning*.

Définition 4.4: Variance et écart-type empiriques

La **variance empirique** (corrigée, ou *sans biais*) d’un échantillon $x_1, \dots, x_n$ mesure la dispersion des données autour de la moyenne :

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2,$$

et l’écart-type empirique est $s = \sqrt{s^2}$.

Remarque 4.2: Pourquoi diviser par $n - 1$?

Diviser par $n - 1$ plutôt que par n — la **correction de Bessel** — corrige le biais systématique qui apparaît lorsque l’on estime la variance d’une population à partir de la moyenne *empirique* plutôt que de la véritable moyenne de la population, inconnue en pratique. Ce point sera justifié formellement à la section 4.3.

Bon à savoir : Standardisation et Batch Normalization

Pourquoi normalise-t-on toujours les données en entrée d’un réseau via $z_i = \frac{x_i - \bar{x}}{s}$? Si les variables ont des variances très différentes (ex : un âge $\in [0, 100]$ et un salaire $\in [10^4, 10^5]$), la fonction de coût aura la forme d’un ravin très étroit (matrice hessienne mal conditionnée, cf. Chapitre 2). La descente de gradient va osciller et diverger. Le **Batch Normalization** étend ce principe : il calcule $\bar{x}$ et s sur chaque mini-batch au cœur même du réseau pour recentrer les activations, ce qui empêche le gradient de s’évanouir dans les zones saturées des fonctions d’activation.

Définition 4.5: Quantiles

Pour $p \in (0, 1)$, le **quantile d’ordre p** d’un échantillon est une valeur q_p telle qu’une proportion p des observations lui soit inférieure ou égale. Les cas particuliers $p = 0,25$,

$p = 0,5$ et $p = 0,75$ sont appelés respectivement premier quartile (Q_1), médiane, et troisième quartile (Q_3).

Définition 4.6: Écart interquartile

L'**écart interquartile** (*interquartile range*, IQR) est $IQR = Q_3 - Q_1$. C'est, à l'instar de la médiane, une mesure de dispersion **robuste** aux valeurs extrêmes, couramment utilisée en *Data Science* pour détecter des valeurs aberrantes : est jugée aberrante toute observation en dehors de l'intervalle $[Q_1 - 1,5 IQR, Q_3 + 1,5 IQR]$.

4.2 Graphiques et visualisations

Le cerveau humain est incapable de déceler des motifs dans un tableau d'un million de lignes. L'exploration visuelle précède et conditionne le succès de toute modélisation.

Définition 4.7: Histogramme

Un **histogramme** partitionne le domaine des valeurs observées en intervalles contigus (*classes*, ou *bins*) et représente, pour chaque classe, un rectangle dont la hauteur est proportionnelle au nombre d'observations tombant dans cette classe. Il fournit une estimation visuelle de la distribution sous-jacente des données (et révèle souvent des bimodalités indiquant des sous-populations cachées).

Définition 4.8: Boîte à moustaches

Une **boîte à moustaches** (*boxplot*) représente, pour un échantillon, une boîte allant de Q_1 à Q_3 (traversée par la médiane), et deux moustaches s'étendant jusqu'aux valeurs les plus extrêmes de l'échantillon restant à l'intérieur de $[Q_1 - 1,5 IQR, Q_3 + 1,5 IQR]$; les observations au-delà sont représentées individuellement comme des points aberrants.

Définition 4.9: Nuage de points

Un **nuage de points** (*scatter plot*) représente chaque observation bivariée (x_i, y_i) par un point dans le plan, permettant de visualiser à l'œil la relation linéaire, monotone, ou plus complexe entre deux variables.

Remarque 4.3: Choisir la bonne visualisation

L'histogramme et la boîte à moustaches renseignent tous deux sur la distribution d'une variable unique, mais de façon complémentaire : l'histogramme révèle la forme complète de la distribution (multimodalité, asymétrie), tandis que la boîte à moustaches résume rapidement position, dispersion et présence de valeurs aberrantes. Elle se prête particulièrement bien à la comparaison des performances de plusieurs modèles lors d'une validation croisée (*Cross-Validation*). Le nuage de points, lui, est l'outil de référence pour explorer les interactions entre caractéristiques (*features*) avant d'ajuster un modèle.

Bon à savoir : Visualiser l'Espace Latent (t-SNE et UMAP)

Le nuage de points classique est limité à la 2D ou 3D. Mais les réseaux de neurones profonds créent des représentations internes (*embeddings*) dans des espaces à des centaines de dimensions. Des algorithmes de réduction de dimensionnalité non-linéaire comme **t-SNE** ou **UMAP** transforment ces dimensions élevées en un nuage de points 2D tout en préservant les voisinages locaux. Si l'UMAP montre des groupes (clusters) bien séparés, cela prouve visuellement que l'IA a compris la structure sémantique des données.

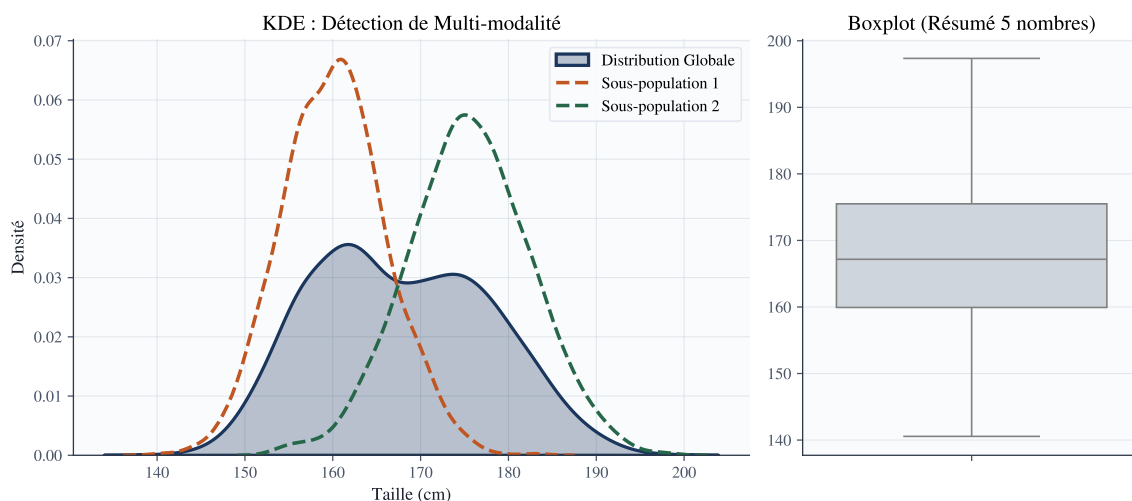


FIGURE 4.2 – L'Analyse Exploratoire des Données (EDA). **À gauche** : L'estimation par noyau de densité (KDE) révèle une bimodalité (deux bosses). Cela indique formellement qu'une variable cachée sépare les données en deux sous-populations (ici, la taille des femmes et des hommes). **À droite** : Le Boxplot offre un résumé extrêmement compact permettant d'identifier immédiatement la médiane et les potentiels *outliers* (les points au-delà des moustaches).

4.3 Échantillonnage et estimation

Définition 4.10: Échantillon aléatoire simple (L'hypothèse I.I.D.)

Un **échantillon aléatoire simple** de taille n issu d'une population de loi $\mathbb{P}$ est une famille $X_1, \dots, X_n$ de variables aléatoires **indépendantes et identiquement distribuées** (i.i.d.) selon $\mathbb{P}$. (En Machine Learning, violer cette condition entre l'entraînement et la production s'appelle le *Data Drift*, ce qui cause l'effondrement des performances du modèle).

Définition 4.11: Estimateur, biais

Un **estimateur** d'un paramètre θ est une statistique $\hat{\theta}_n = g(X_1, \dots, X_n)$, fonction de l'échantillon, censée approcher θ (Un modèle d'IA est un estimateur complexe). Son **biais** est $\text{Biais}(\hat{\theta}_n) = \mathbb{E}[\hat{\theta}_n] - \theta$; l'estimateur est dit **sans biais** si $\text{Biais}(\hat{\theta}_n) = 0$ pour tout θ .

Propriété 4.1: La moyenne empirique est un estimateur sans biais

Si $X_1, \dots, X_n$ sont i.i.d. d'espérance μ , alors $\mathbb{E}[\bar{X}_n] = \mu$: la moyenne empirique est un estimateur sans biais de l'espérance. On montre de même, en utilisant la correction de Bessel introduite à la section 4.1, que $\mathbb{E}[s^2] = \sigma^2$: diviser par $n - 1$ rend précisément la variance empirique sans biais.

Théorème 4.1: Loi (faible) des grands nombres

Soit $X_1, X_2, \dots$ une suite de variables aléatoires i.i.d. d'espérance finie μ . Alors, pour tout $\varepsilon > 0$,

$$\mathbb{P}(|\bar{X}_n - \mu| > \varepsilon) \xrightarrow[n \rightarrow \infty]{} 0.$$

Le moteur du Deep Learning : Le Mini-Batch SGD

Le calcul exact du gradient d'un réseau sur des millions d'images est impossible (saturation mémoire). L'algorithme de **Descente de Gradient Stochastique par mini-lots (Mini-Batch SGD)** tire un échantillon aléatoire de 64 images, calcule le gradient empirique, et met à jour les poids. La **Loi des Grands Nombres** est la garantie théorique absolue que, bien que ce calcul soit bruité à chaque étape, il pointe mathématiquement, en moyenne, dans la direction exacte du vrai gradient de la fonction de perte !

Théorème 4.2: Théorème central limite (TCL)

Soit $X_1, X_2, \dots$ une suite de variables aléatoires i.i.d. d'espérance μ et de variance finie $\sigma^2 > 0$. Alors

$$\sqrt{n} \frac{\bar{X}_n - \mu}{\sigma} \xrightarrow[n \rightarrow \infty]{d} \mathcal{N}(0, 1),$$

c'est-à-dire que la loi de l'erreur moyenne converge vers la loi normale centrée réduite, **quelle que soit la loi sous-jacente des X_i** .

Remarque 4.4: Portée du théorème central limite

Ce résultat spectaculaire explique pourquoi la loi normale apparaît si fréquemment en pratique : la somme d'un grand nombre de contributions aléatoires (ex : les erreurs d'un modèle) a toujours tendance à se distribuer selon une cloche. C'est le TCL qui justifie théoriquement la construction des intervalles de confiance de la section suivante.

Bon à savoir : Le Bootstrap et le Random Forest

Que faire si l'échantillon est trop petit pour appliquer le TCL ? On utilise le **Bootstrap** : on tire informatiquement des milliers de sous-échantillons *avec remise* depuis notre jeu de données original pour simuler empiriquement la variance de notre estimateur. C'est le principe fondateur du **Random Forest** (Forêts Aléatoires). En entraînant des centaines d'arbres de décision sur des échantillons Bootstrapés, et en moyennant leurs réponses (**Bagging** = *Bootstrap Aggregating*), on réduit mathématiquement la variance du modèle global sans augmenter son biais.

4.4 Statistiques inférentielles : L'Évaluation Rigoureuse

Comparer deux IA sur la base d'une simple précision (ex : 92% vs 91%) sans outils inférentiels est une faute scientifique : l'écart peut être purement dû au hasard de l'échantillon de test.

Définition 4.12: Intervalle de confiance

Un **intervalle de confiance** de niveau $1 - \alpha$ pour un paramètre θ (ex : la précision réelle d'un modèle) est un intervalle aléatoire $[\hat{\theta}_n^-, \hat{\theta}_n^+]$, construit à partir de l'échantillon, tel que

$$\mathbb{P}(\theta \in [\hat{\theta}_n^-, \hat{\theta}_n^+]) \geq 1 - \alpha.$$

Propriété 4.2: Intervalle de confiance pour une moyenne

Si σ est connu (ou n suffisamment grand pour invoquer le théorème 4.2), un intervalle de confiance de niveau $1 - \alpha$ pour μ est

$$\left[\bar{X}_n - z_{1-\alpha/2} \frac{\sigma}{\sqrt{n}}, \bar{X}_n + z_{1-\alpha/2} \frac{\sigma}{\sqrt{n}} \right],$$

où $z_{1-\alpha/2}$ est le quantile d'ordre $1 - \alpha/2$ de la loi $\mathcal{N}(0, 1)$ (par exemple $z_{0,975} \approx 1,96$ pour $\alpha = 0,05$). *Remarque : La précision s'améliore au rythme de $\sqrt{n}$. Pour diviser l'incertitude par 2, il faut tester le modèle sur 4 fois plus de données.*

Définition 4.13: Hypothèses nulle et alternative

Un **test d'hypothèse** (essentiel dans l'**A/B Testing**) confronte une **hypothèse nulle** H_0 (l'absence d'effet, par exemple : "L'IA B n'est pas meilleure que l'IA A") à une **hypothèse alternative** H_1 , à partir d'un échantillon.

Définition 4.14: Statistique de test, p -valeur

Une **statistique de test** T est une fonction de l'échantillon dont la loi est connue sous H_0 . La **p -valeur** est la probabilité, *calculée sous H_0* , d'observer une statistique de test au moins aussi extrême que celle effectivement observée :

$$p\text{-valeur} = \mathbb{P}_{H_0}(|T| \geq |t_{\text{obs}}|).$$

Propriété 4.3: Décision au seuil α

On rejette H_0 au profit de H_1 si la p -valeur est inférieure au seuil de signification α fixé à l'avance (typiquement $\alpha = 0,05$).

Remarque 4.5: Erreurs de première et deuxième espèce

Rejeter H_0 alors qu'elle est vraie est une **erreur de première espèce** (Faux Positif), de probabilité contrôlée par α . Ne pas rejeter H_0 alors qu'elle est fautive est une **erreur de deuxième espèce**, de probabilité notée β ; $1 - \beta$ est la **puissance** du test. Une p -valeur petite ne mesure en aucun cas l'ampleur d'un effet, seulement la compatibilité des

données avec H_0 .

Exemple 4.1: Test t de Student pour une moyenne

Pour tester $H_0 : \mu = \mu_0$ contre $H_1 : \mu \neq \mu_0$ à partir d'un échantillon $X_1, \dots, X_n$ de moyenne $\bar{X}_n$ et d'écart-type empirique s , on utilise la statistique

$$T = \frac{\bar{X}_n - \mu_0}{s/\sqrt{n}},$$

qui suit, sous H_0 et sous hypothèse de normalité des X_i , une loi de Student à $n - 1$ degrés de liberté. La p -valeur s'obtient en comparant la valeur observée t_{obs} aux quantiles de cette loi.

Le Piège de l'IA : Le Problème des Tests Multiples (P-Hacking)

Vous lancez une recherche d'hyperparamètres (*Grid Search*) pour évaluer 100 architectures de réseaux différentes sur votre jeu de test. Vous choisissez celle avec la plus petite p -valeur face au modèle de base. **C'est une erreur mortelle.** Si vous testez 100 modèles totalement inefficaces, par pure fluctuation statistique (100×0.05), environ 5 d'entre eux obtiendront une p -valeur < 0.05 et sembleront excellents par hasard! Pour éviter de déployer des modèles sur-ajustés, il faut appliquer la **Correction de Bonferroni** (diviser le seuil α par le nombre de tests N) et toujours conserver un ultime jeu de test aveugle évalué *une seule fois*.

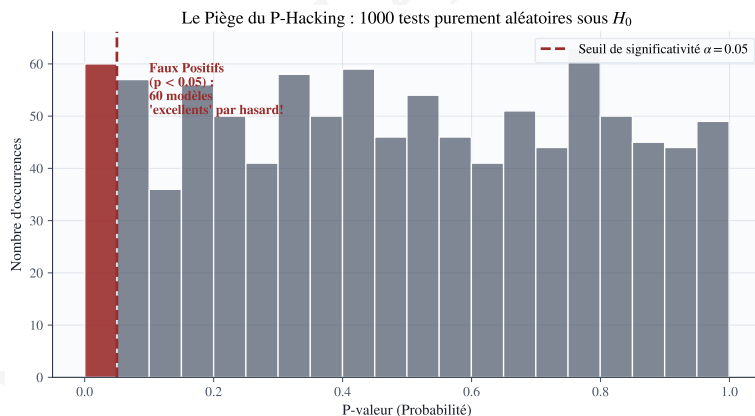


FIGURE 4.3 – Le piège statistique des tests multiples (*P-Hacking*). Sur 1 000 tests A/B comparant deux modèles générés de manière *purement aléatoire* (l'hypothèse nulle H_0 est donc strictement vraie), environ 5% des tests (en rouge) tombent naturellement sous le seuil $\alpha = 0.05$. Si l'on teste aveuglément des dizaines d'hyperparamètres, on finira par sélectionner une "fausse découverte" statistique. La correction de Bonferroni est donc impérative.

4.5 Corrélation et notions de causalité : La Frontière de l'IA

Définition 4.15: Covariance

La **covariance** empirique entre deux variables $x_1, \dots, x_n$ et $y_1, \dots, y_n$ évalue leur variation conjointe :

$$s_{xy} = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}).$$

Définition 4.16: Coefficient de corrélation de Pearson

Le **coefficient de corrélation de Pearson** est

$$r_{xy} = \frac{s_{xy}}{s_x s_y} \in [-1, 1],$$

où s_x, s_y sont les écarts-types empiriques. Il mesure l'intensité d'une relation **linéaire** entre les deux variables.

Remarque 4.6: Corrélation n'implique pas causalité

Une corrélation élevée entre deux variables X et Y peut provenir de trois scénarios : X influence causalement Y , Y influence causalement X , ou une **variable confondante** Z influence à la fois X et Y sans lien causal direct entre elles.

Exemple 4.2: Corrélation fallacieuse

Le nombre de noyades et les ventes de crème glacée sont fortement corrélés au cours de l'année : aucune des deux ne cause l'autre, la variable confondante étant la **température**, qui augmente à la fois la fréquentation des plages (donc les noyades) et la consommation de crème glacée.

Le mur du Machine Learning Actuel : L'Apprentissage Causal

Les IA d'aujourd'hui (Deep Learning, Transformers) sont de puissants extracteurs de corrélations fallacieuses. Si 90% des photos de vaches dans votre dataset ont un fond de pâturage vert, un réseau convolutif apprendra à détecter l'herbe verte pour dire "Vache". Présentez-lui une vache sur une plage, l'IA se trompera lamentablement. Elle s'est accrochée à une corrélation au lieu d'apprendre la structure *causale* de l'animal. Le chercheur Judea Pearl a théorisé que l'IA doit s'élever sur 3 niveaux :

1. **Association (L'IA actuelle)** : $\mathbb{P}(y | x)$ - L'observation passive.
2. **Intervention (A/B Testing)** : $\mathbb{P}(y | do(x))$ - Que se passe-t-il si je force l'action, détruisant ainsi la variable confondante ?
3. **Contrefactuel (Raisonnement)** : Que se serait-il passé si j'avais agi différemment ?

L'Inférence Causale est le Saint Graal permettant aux modèles de généraliser hors de leur distribution d'entraînement (*Out-Of-Distribution*).

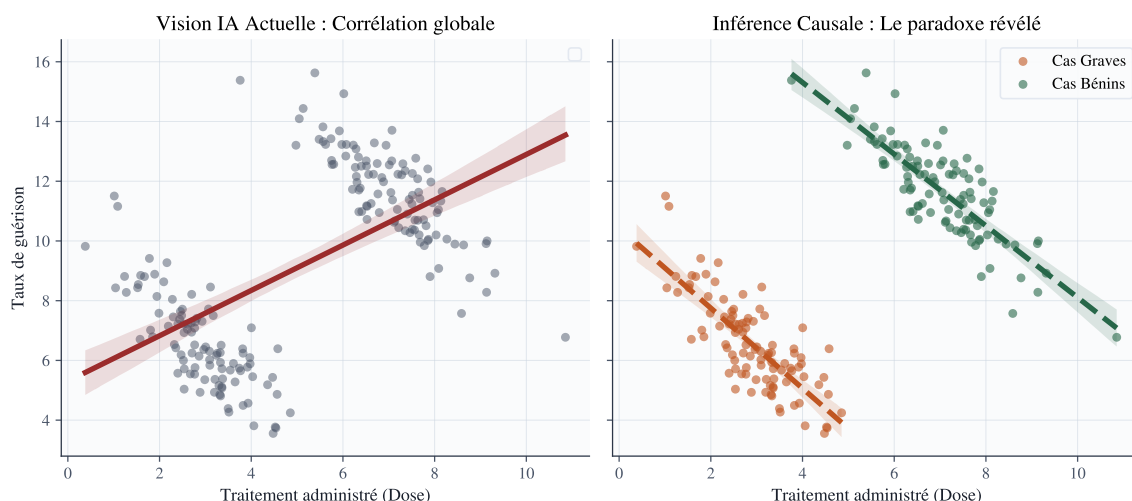


FIGURE 4.4 – Le Paradoxe de Simpson et la Variable Confondante. À gauche (**Vision IA actuelle**) : Si l’on ne regarde que la corrélation globale, l’IA en conclut que plus on augmente le traitement, plus le taux de guérison grimpe (Tendance positive rouge). À droite (**Inférence Causale**) : Si l’on sépare la population selon la variable confondante (La gravité de la maladie d’origine), on observe que la vraie tendance causale est *négative* pour chaque sous-groupe. L’IA naïve aurait prescrit un traitement mortel.

Points clés du chapitre

Essentiels à retenir

- Les **statistiques descriptives** (moyenne, médiane, variance) résument un jeu de données. La moyenne est liée à l’erreur L_2 (sensible aux outliers) tandis que la médiane minimise l’erreur L_1 (robuste).
- Histogramme, boîte à moustaches et nuage de points sont les visualisations de référence. Combinés à **t-SNE/UMAP**, ils permettent de sonder les représentations internes des réseaux de neurones.
- La **loi des grands nombres** et le **théorème central limite** justifient mathématiquement la descente de gradient stochastique (SGD) et la création d’intervalles de confiance.
- Les **intervalles de confiance** et les **tests d’hypothèses** (p -valeur) quantifient la fiabilité d’un modèle. Il faut se méfier du *P-Hacking* en appliquant la correction de Bonferroni.
- Une forte **corrélation** n’implique jamais une relation de **causalité**. L’IA de demain devra intégrer l’inférence causale pour cesser d’exploiter des variables confondantes.

Exercices et Travaux Pratiques (TP)

1. Pour l'échantillon $\{2, 4, 4, 4, 5, 5, 7, 9\}$, calculer la moyenne, la médiane, le mode, la variance empirique corrigée et l'écart interquartile. Démontrez que la médiane est plus robuste si on remplace 9 par 900.
2. Expliquer, à l'aide de la section 4.3, pourquoi la moyenne empirique reste un estimateur sans biais de μ quelle que soit la taille n de l'échantillon, alors que la précision de l'estimation, elle, s'améliore avec n (Lien avec le TCL).
3. Un test de Student donne une p -valeur de 0,03 pour $H_0 : \mu = 0$ contre $H_1 : \mu \neq 0$, au seuil $\alpha = 0,05$. Formuler la décision statistique correspondante, puis expliquer pourquoi cette p -valeur ne permet pas, à elle seule, de conclure à l'importance pratique (magnitude) de l'effet mesuré.
4. On observe une corrélation de $r_{xy} = 0,85$ entre le nombre de pompiers dépêchés sur un incendie et les dégâts matériels causés. Discuter, à la lumière de la section 4.5, si l'on peut en conclure que les pompiers aggravent les dégâts (Identifiez la variable confondante).
5. **(TP Python)** À l'aide de Pandas, charger un jeu de données tabulaire complet (ex : Titanic ou Diamonds).
 - (a) Calculez les statistiques descriptives (`DataFrame.describe`).
 - (b) Produisez, avec Seaborn, un histogramme lissé (KDE), une boîte à moustaches pour détecter les outliers, et une matrice de corrélation sous forme de carte de chaleur (*heatmap*).
 - (c) Implémentez un test de Student (`scipy.stats.ttest_ind`) pour vérifier si la différence de prix moyen (ou survie) entre deux sous-groupes est statistiquement significative.

Chapitre 5

Mathématiques discrètes

Introduction générale

Les chapitres précédents ont mobilisé les mathématiques du continu : calcul différentiel pour l'optimisation, algèbre linéaire pour les espaces latents, lois de probabilité à densité pour l'incertitude. Cependant, l'ordinateur qui exécute l'Intelligence Artificielle est, par nature, une machine strictement finie et discrète.

Tout modèle de *Machine Learning* est, en dernière instance, exécuté par un algorithme, c'est-à-dire une suite finie d'opérations. Il manipule des structures de données profondément discrètes : des ensembles d'images, des vocabulaires de mots découpés en *tokens*, des arbres de décision, ou encore des graphes sociaux. Les mathématiques discrètes fournissent le langage rigoureux nécessaire pour raisonner sur l'architecture de ces objets et sur l'efficacité des algorithmes qui les manipulent.

Ce chapitre, qui clôt l'unité d'enseignement Mathématiques Fondamentales, commence par la **logique propositionnelle et des prédicats**. Au-delà des simples conditions if/else, elle forme la base de l'IA Neuro-Symbolique. On revoit ensuite le vocabulaire des **ensembles, relations et fonctions** sous un prisme nouveau : celui de l'inversibilité des réseaux de neurones (bijections) et de l'apprentissage contrastif (relations d'équivalence).

La **combinatoire** (dénombrement) nous permettra de toucher du doigt le pire cauchemar de l'IA : la *Malédiction de la dimensionnalité*. On introduira ensuite la **théorie des graphes**, car un réseau de neurones n'est finalement rien d'autre qu'un graphe d'opérations mathématiques. Le chapitre se conclut en liant ces notions à la **complexité algorithmique** (le fameux "Grand O"), clef de voûte expliquant pourquoi l'entraînement d'un *Transformer* comme ChatGPT coûte des millions de dollars.

Objectif

Objectif : Fournir les fondements logiques, combinatoires et graphiques du raisonnement algorithmique pour comprendre les limites computationnelles de l'Intelligence Artificielle.

Masse horaire : 2 h de cours • 1 h de TD/TP.

● Outils & bibliothèques

Python, NetworkX

5.1 Logique propositionnelle et des prédicats

L'IA ne se limite pas aux statistiques. Dans des domaines critiques (véhicules autonomes, médecine), il est crucial de pouvoir *prouver* formellement qu'un système respecte des règles de sécurité.

Définition 5.1: Proposition

Une **proposition** est un énoncé mathématique ou informatique auquel on peut attribuer une valeur de vérité binaire : **Vrai** (1) ou **Faux** (0), sans ambiguïté.

Définition 5.2: Connecteurs logiques

À partir de propositions simples p, q , on construit des systèmes complexes via des "portes" logiques :

$\neg p$ (négation, NON), $p \wedge q$ (conjonction, ET), $p \vee q$ (disjonction, OU)

$p \Rightarrow q$ (implication, SI p ALORS q), $p \Leftrightarrow q$ (équivalence)

En termes simples : L'implication $p \Rightarrow q$ est fausse **uniquement** si la promesse est brisée (c'est-à-dire si p est vrai mais que q est faux). Dans tous les autres cas (notamment si p est faux au départ), l'implication est considérée comme vraie (*ex falso quodlibet* : du faux, on peut déduire n'importe quoi).

Propriété 5.1: Lois de De Morgan

Les lois de De Morgan permettent d'inverser des conditions algorithmiques complexes. Pour toutes propositions p, q :

$$\neg(p \wedge q) \equiv (\neg p) \vee (\neg q), \quad \neg(p \vee q) \equiv (\neg p) \wedge (\neg q).$$

Application Python : Dire `not (x > 0 and y < 5)` est strictement équivalent, pour le compilateur, à `(x <= 0) or (y >= 5)`.

Définition 5.3: Prédicat, quantificateurs

Un **prédicat** $P(x)$ est une fonction qui renvoie un booléen (Vrai/Faux) selon la valeur du paramètre $x \in E$. Les **quantificateurs** permettent de tester tout un ensemble :

$\forall x \in E, P(x)$ (« pour TOUT x , $P(x)$ est vrai »)

$\exists x \in E, P(x)$ (« il EXISTE au moins un x tel que $P(x)$ est vrai »)

Négation : $\neg(\forall x \in E, P(x)) \equiv \exists x \in E, \neg P(x)$. (Pour prouver que "tous les cygnes sont blancs" est faux, il suffit de trouver *un seul* cygne noir).

La Frontière de l'IA : L'IA Neuro-Symbolique

Le Deep Learning classique est purement probabiliste (cf. Chapitre 3). S'il prédit qu'un feu est vert à 99%, la voiture avance. Mais que se passe-t-il si un hacker ajoute du bruit sur l'image (attaque adversarienne)? La recherche moderne se tourne vers l'**IA Neuro-Symbolique**. Elle fusionne les réseaux de neurones (pour extraire des concepts des pixels) avec des moteurs de **Règles Logiques** dures ($\forall \text{ piéton} \in \text{passage_clouté} \Rightarrow \neg \text{accélérer}$). Cela permet de créer des IA capables de raisonner mathématiquement et d'être auditées formellement.

5.2 Ensembles, relations, fonctions : Au cœur des Architectures

Définition 5.4: Opérations ensemblistes

Pour A, B des sous-ensembles d'un espace global E (ex : des groupes d'images) :

- **Union** $A \cup B$: les éléments dans A **ou** B (ou les deux).
- **Intersection** $A \cap B$: les éléments dans A **et** B simultanément.
- **Différence** $A \setminus B$: les éléments de A qui ne sont pas dans B .
- **Produit cartésien** $A \times B$: l'ensemble de toutes les paires possibles (a, b) .

Définition 5.5: Fonction, injection, surjection, bijection

Une **fonction** (ou *mapping*) $f: A \rightarrow B$ associe à chaque élément de l'espace d'entrée A un unique élément dans l'espace de sortie B . Un réseau de neurones est fondamentalement une fonction mathématique complexe.

- **Injective** : Aucun point d'arrivée n'a deux points de départ différents. $\forall a, a', f(a) = f(a') \Rightarrow a = a'$. (En ML : Il n'y a pas de "collision", aucune information de l'entrée n'est perdue ou superposée).
- **Surjective** : L'espace d'arrivée est entièrement couvert. $\forall b \in B, \exists a \in A, f(a) = b$. (En IA générative : Le modèle est capable de générer n'importe quelle image possible du monde réel).
- **Bijective** : Injective ET Surjective. La transformation est parfaite et strictement inversible via f^{-1} .

Interprétation Deep Learning : Normalizing Flows (Bijections)

La plupart des réseaux de neurones (ex : ResNet) réduisent la dimension d'une image 256×256 en un vecteur de taille 1000. Ils ne sont donc *pas injectifs* (perte d'information géométrique, irréversible). Les **Normalizing Flows** sont une classe révolutionnaire de modèles génératifs. Leur architecture est contrainte mathématiquement pour être strictement **Bijective**. Ils transforment une distribution complexe (des visages) en une distribution simple (loi Normale) de manière réversible f^{-1} . Grâce à cette bijection, on peut générer de nouveaux visages parfaits avec une formule de probabilité mathématiquement exacte, sans l'instabilité des GANs!

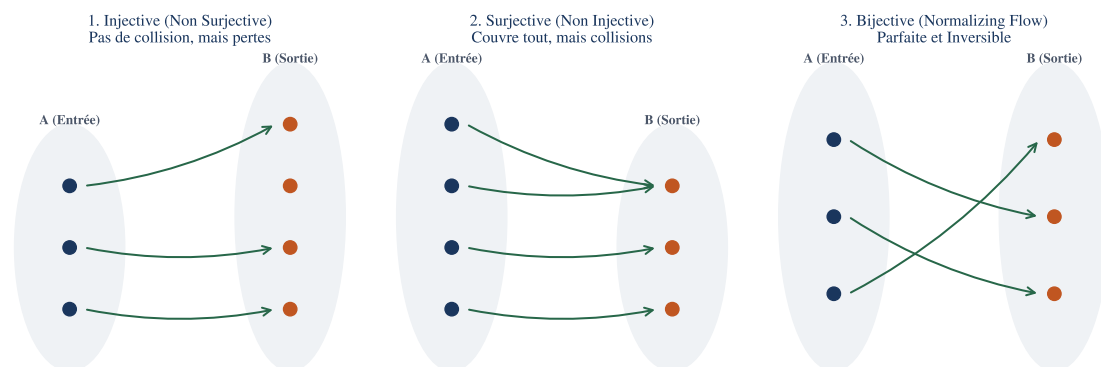


FIGURE 5.1 – Classification topologique des fonctions en *Machine Learning*. (1) Une fonction **Injective** ne superpose jamais deux entrées (aucune collision), mais peut ignorer une partie de l'espace d'arrivée. (2) Une fonction **Surjective** génère tous les résultats possibles, mais détruit de l'information (réduction de dimension des réseaux convolutifs). (3) Une fonction **Bijective** relie chaque entrée à une sortie unique et est strictement inversible. Les modèles génératifs de pointe (*Normalizing Flows*) exploitent cette bijection pour mapper parfaitement l'espace du bruit (loi normale) vers l'espace des données (images réelles).

Définition 5.6: Relations et Classes d'équivalence

Une relation R entre éléments est une **relation d'équivalence** si elle est réflexive (xRx), symétrique (si xRy alors yRx) et transitive (si xRy et yRz , alors xRz). Elle découpe l'espace en "classes d'équivalence" (des groupes isolés).

Remarque 5.1: L'Apprentissage Contrastif (Contrastive Learning)

L'un des plus grands succès récents de l'IA auto-supervisée (SimCLR, CLIP) repose sur la notion de relation d'équivalence. On prend une image de chien x , on lui applique des rotations ou du flou pour créer x' . Le réseau est forcé de créer une relation d'équivalence $f(x) \approx f(x')$ dans l'espace latent (ils appartiennent à la même classe sémantique), tout en éloignant (relation d'ordre/distance) les images de chats y .

5.3 Combinatoire : Dénombrement et Explosion Dimensionnelle

La combinatoire est l'art de compter sans énumérer.

Propriété 5.2: Principe multiplicatif

Si un choix A offre n_1 possibilités, et qu'un choix B indépendant offre n_2 possibilités, le système combiné offre $n_1 \times n_2$ possibilités.

Définition 5.7: Arrangements et Combinaisons

Combien y a-t-il de façons de tirer k objets parmi un sac de n objets ?

- **L'ordre compte (Arrangement)** : $A_n^k = \frac{n!}{(n-k)!}$. (Exemple : Faire un podium 1er, 2e, 3e parmi 10 athlètes. Les permutations importent).
- **L'ordre s'en fiche (Combinaison)** : Le fameux **Coefficient Binomial** $\binom{n}{k} = \frac{n!}{k!(n-k)!}$. (Exemple : Choisir une équipe de 3 athlètes parmi 10. Le rôle de chacun importe peu, seul le groupe final compte).

Le Pire Cauchemar du Machine Learning : L'Explosion Combinatoire

Supposons que vous cherchiez les meilleurs hyperparamètres d'un réseau : vous testez 5 *learning rates*, 5 *batch sizes*, 5 *fonctions d'activation*, et 5 *nombre de couches*. Par le principe multiplicatif, cela fait $5 \times 5 \times 5 \times 5 = 625$ entraînements à faire. Si on ajoute juste 3 autres paramètres à régler, on passe à $5^7 = 78\,125$ modèles à entraîner ! L'univers de recherche croît de façon exponentielle. C'est la **Malédiction de la Dimensionnalité** (*Curse of Dimensionality*). C'est pourquoi la recherche en grille (*Grid Search*) est remplacée par la recherche aléatoire (*Random Search*) ou l'Optimisation Bayésienne, qui naviguent intelligemment dans ce vaste espace discret.

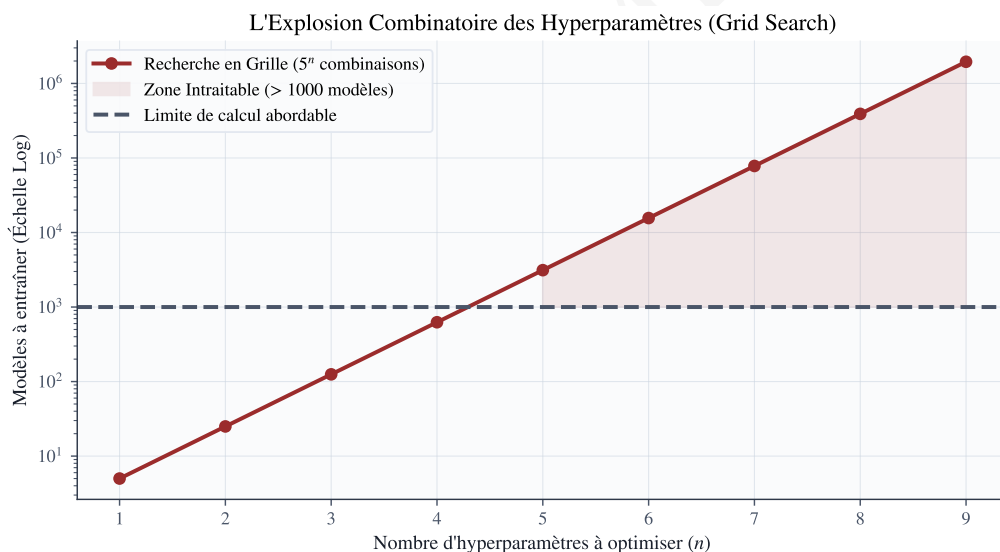


FIGURE 5.2 – La Malédiction de la Dimensionnalité (*Curse of Dimensionality*). En combinatoire, si l'on effectue une recherche en grille (*Grid Search*) en testant 5 valeurs possibles pour n hyperparamètres, le nombre de modèles à entraîner s'envole de manière exponentielle (5^n). Au-delà de 4 paramètres, le calcul exhaustif devient physiquement intraitable, justifiant l'usage de méthodes stochastiques (Recherche aléatoire) ou probabilistes (Optimisation Bayésienne).

5.4 Théorie des graphes : La structure sous-jacente de l'IA

En IA moderne, les données tabulaires (Excel) laissent la place à des données purement relationnelles : molécules, réseaux sociaux, trafic routier.

Définition 5.8: Graphe, Degré, Voisinage

Un **graphe** (non orienté) $G = (V, E)$ est un ensemble de **nœuds/sommets** V (les entités, ex : des personnes) reliés par des **arêtes** E (les relations, ex : "est ami avec"). Le **voisinage** d'un sommet v est la liste des sommets directement connectés à lui. Le **degré** $\deg(v)$ est le nombre de ses voisins.

Propriété 5.3: Lemme des poignées de main

Dans toute soirée (ou tout graphe), la somme des poignées de main échangées (les degrés de chaque nœud) est exactement égale au double du nombre de poignées de main réelles (les arêtes). $\sum_{v \in V} \deg(v) = 2|E|$.

Définition 5.9: Chemin, Connexité et Arbre

Un graphe est **connexe** s'il n'est pas fragmenté en plusieurs îlots isolés (on peut aller de n'importe quel nœud à n'importe quel autre). Un **arbre** est un graphe connexe sans aucun cycle (pas de boucle fermée). Les *Arbres de Décision* en Machine Learning en sont l'application directe.

Propriété 5.4: Parcours de Graphes (BFS et DFS)

Pour explorer un graphe algorithmiquement :

- **Parcours en largeur (BFS - Breadth-First Search)** : On explore en cercles concentriques. On visite tous les amis directs, puis les amis d'amis. Utilise une structure de file (FIFO). Sert à trouver le *plus court chemin*.
- **Parcours en profondeur (DFS - Depth-First Search)** : On s'enfonce le plus loin possible dans une branche, jusqu'à un cul-de-sac, puis on fait marche arrière (*backtracking*). Utilise une pile (LIFO).

Au cœur de PyTorch : Les Graphes de Calcul et GNN

1. **Autograd (Rétropropagation)** : Lorsque vous écrivez $z = x \times y + 2$ en PyTorch, le framework construit dynamiquement sous le capot un Graphe Orienté Acyclique (DAG). Les nœuds sont les tenseurs $(x, y, 2)$ et les arêtes sont les opérations $(\times, +)$. Le parcours DFS inversé de ce graphe est *exactement* ce qui permet de calculer le gradient via la Règle de la Chaîne (Chapitre 1)!

2. **Graph Neural Networks (GNN)** : Au lieu de traiter des pixels indépendants, les GNN traitent des graphes. Pour mettre à jour l'embedding d'un nœud v (ex : atome de carbone), l'IA récupère les embeddings de son *voisinage* immédiat $N(v)$ et les agrège (Mécanisme de *Message Passing*). Cela a permis la découverte de nouveaux antibiotiques (AlphaFold/Halicin).

5.5 Complexité Algorithmique : Le Prix de l'Intelligence

Les mathématiques ont prouvé qu'un réseau de neurones pouvait approximer n'importe quelle fonction (Théorème d'approximation universelle). La question n'est donc plus "Peut-on le faire?", mais "En combien de temps / Avec combien d'énergie?".

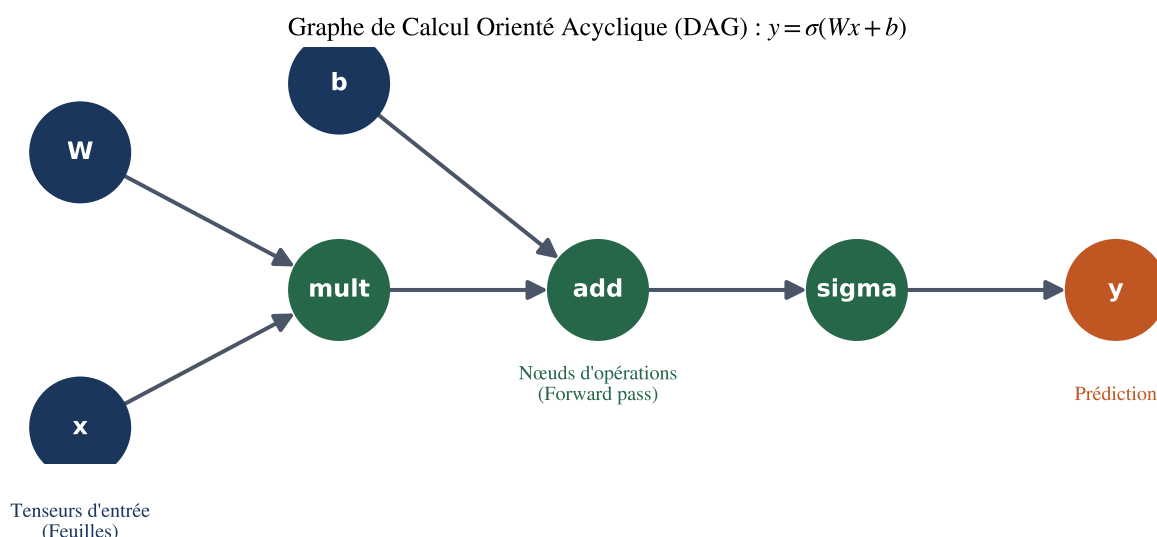


FIGURE 5.3 – Modélisation d’une équation mathématique sous forme de **Graphe Orienté Acyclique (DAG)**. C’est exactement la structure de données discrète générée dynamiquement par PyTorch lors de l’entraînement. La passe avant (*Forward Pass*) correspond à un tri topologique du graphe de la gauche vers la droite. Lors de la *rétropropagation*, l’algorithme parcourt ce graphe à l’envers (DFS) pour dériver la fonction de perte en appliquant la règle de la chaîne à chaque arête.

Définition 5.10: Notation de Landau (Le Grand O)

On dit que le temps d’exécution $f(n)$ d’un algorithme est $O(g(n))$ s’il est proportionnel à $g(n)$ lorsque la taille des données n devient très grande. On ignore les constantes (ex : $3n^2 + 5n + 10$ devient simplement $O(n^2)$). Cette notation décrit **comment le temps de calcul explose quand la donnée grandit**.

Exemple 5.1: La hiérarchie des complexités (Du meilleur au pire)

- $O(1)$ (**Temps constant**) : Accéder à la valeur d’un dictionnaire Python par sa clé. Le temps est identique pour 10 ou 1 milliard d’entrées.
- $O(\log n)$ (**Logarithmique**) : Recherche dichotomique dans une liste **triée**. Très efficace.
- $O(n)$ (**Linéaire**) : Lire chaque élément d’une liste un par un. (Temps double si la liste double).
- $O(n^3)$ (**Cubique**) : Produit naïf de deux matrices $n \times n$. Multiplier la taille de l’image par 2 multiplie le temps de calcul par 8 ! D’où l’invention matérielle des cœurs Tensor (TPU/GPU) qui parallélisent cette opération.

La crise de l'IA Moderne : Le Goulet d'étranglement $O(n^2)$ de l'Attention

Pourquoi ChatGPT limite-t-il ses réponses et le texte que vous lui envoyez (la fenêtre de contexte)? L'architecture Transformer repose sur le mécanisme de *Self-Attention*. Pour comprendre le contexte, chaque *token* (mot) de la phrase doit se comparer mathématiquement à **tous les autres tokens** de la phrase. Pour une séquence de longueur N , cela requiert $N \times N$ comparaisons. La complexité en mémoire et en temps est strictement $O(N^2)$. Si on passe d'un contexte de 4 000 mots à 128 000 mots (facteur 32), la RAM nécessaire sur la carte graphique (VRAM) est multipliée par 1024! Réduire la complexité quadratique de l'attention en $O(N \log N)$ ou $O(N)$ (Mamba, Linear Attention) est la quête absolue de la recherche actuelle en IA.

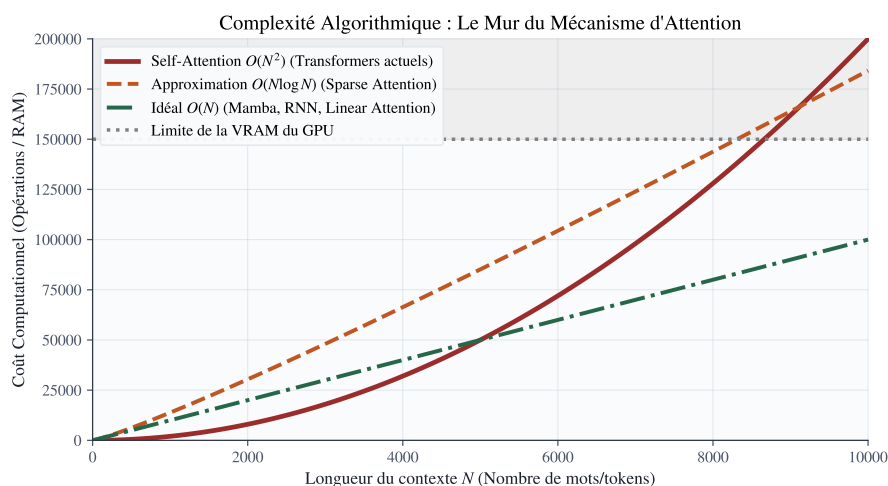


FIGURE 5.4 – Comparaison des complexités algorithmiques (Grand O) face à l'allongement du contexte en Intelligence Artificielle. Le mécanisme d'*Attention* des Transformers exige que chaque mot se compare à tous les autres, générant une complexité spatio-temporelle en $O(N^2)$ (en rouge). Cette courbe heurte très rapidement la limite physique de la VRAM des processeurs graphiques (GPU). La recherche de modèles sub-quadratiques (Mamba en $O(N)$) est l'enjeu majeur pour traiter des livres entiers ou de l'ADN.

Points clés du chapitre

Essentiels à retenir

- La **logique propositionnelle** est la base du raisonnement rigoureux et déterministe. Elle refait surface en IA sous forme de réseaux neuro-symboliques pour garantir la sécurité et la certitude (Trustable AI).
- L'étude des **Fonctions** (Injections, Surjections) n'est pas qu'un jeu d'ensembles : elle détermine si un modèle IA perd de l'information (réduction de dimension) ou s'il est parfaitement réversible (*Normalizing Flows* pour la génération).
- La **Combinatoire** explique l'**Explosion Dimensionnelle**. Le nombre de combinaisons d'hyperparamètres croît si vite que la recherche systématique est un échec computationnel.
- Un **Graphe** (sommets, arêtes) modélise la réalité relationnelle. Le *Machine Learning* utilise les graphes de deux manières : **en surface** (GNNs pour analyser des molécules ou réseaux sociaux) et **sous le capot** (Les Graphes de Calcul Orientés Acycliques, ou DAG, pour exécuter l'algorithme d'Autograd).
- La notation de complexité **Grand O** régit les limites du possible. La complexité en $O(N^2)$ des Transformers explique pourquoi l'accès à une IA conversationnelle avec contexte infini nécessite des ruptures architecturales profondes.

Exercices et Travaux Pratiques (TP)

Partie I : Exercices Théoriques (Logique, Combinatoire et Arbres)

- Preuve par Table de Vérité (Lois de De Morgan) :**
Prouvez la première loi $\neg(p \wedge q) \equiv (\neg p) \vee (\neg q)$ en construisant une table de vérité exhaustive pour toutes les 4 combinaisons possibles de $(p, q) \in \{V, F\}^2$.
- Inversibilité d'un réseau (Fonction Bijective) :**
Soit une fonction d'activation ReLU définie par $f(x) = \max(0, x)$.
(a) Cette fonction est-elle injective sur $\mathbb{R}$? Justifiez en trouvant un contre-exemple de collision.
(b) Une architecture uniquement basée sur ReLU peut-elle être utilisée pour créer un modèle génératif de type *Normalizing Flow* (strictement réversible)? Pourquoi?
- Explosion combinatoire et NLP (Traitement du Langage Naturel) :**
Vous construisez un modèle de langage simpliste basé sur des trigrammes (séquences de 3 mots). Si votre vocabulaire contient 100 000 mots distincts. Combien d'arrangements possibles de 3 mots (*avec répétition* des mots possible) existent-ils? Exprimez ce nombre en puissance de 10. Comprenez-vous pourquoi stocker une matrice de probabilité pour chaque séquence est impossible sans plongement dense (*Embeddings*)?
- Théorie des Graphes et Topologie réseau :**
Soit G un **Arbre** à n sommets. Démontrez (par récurrence ou en argumentant sur la construction du graphe sommet par sommet à partir d'une racine) qu'un arbre possède rigoureusement $n - 1$ arêtes, prouvant ainsi sa nature de réseau minimalement connexe.

Partie II : Travaux Pratiques sous Python (NetworkX et Complexité)

Dans ce TP, nous allons utiliser la bibliothèque NetworkX pour simuler un Graphe de Calcul (comme le fait PyTorch) et utiliser le module `time` pour observer la complexité empirique d'un algorithme.

- Création et Parcours d'un Graphe de Calcul :**
Importez `networkx` as `nx`.
(a) Créez un graphe orienté $G = nx.DiGraph()$.
(b) Ajoutez des nœuds et des arêtes directionnelles pour modéliser l'équation $y = \sigma(Wx + b)$. Les nœuds seront : 'x', 'W', 'b', 'mult' (multiplication), 'add' (addition), 'sigma' (activation), 'y'. (Tracez les arêtes logiques du flux de données).
(c) Visualisez le graphe avec `nx.draw(G, with_labels=True, node_color='lightblue')`.
(d) Utilisez la fonction de tri topologique (`list(nx.topological_sort(G))`). Quel est le but de ce tri? En quoi est-il crucial pour exécuter la "Passe Avant" (*Forward Pass*) d'un réseau dans le bon ordre mathématique?
- Le Mur Quadratique $O(N^2)$ (Benchmark empirique) :**
Nous allons prouver expérimentalement le mur de complexité des Transformers. Écrivez une fonction naïve `self_attention_mock(N)` qui crée une liste de taille N , puis contient une double boucle `for i in range(N): for j in range(N):` pass pour simuler la comparaison de chaque *token* avec tous les autres.

- (a) Utilisez `time.time()` pour mesurer la durée d'exécution pour $N = [1000, 2000, 4000, 8000]$.
- (b) Tracez un graphique de la durée d'exécution (en secondes) en fonction de N via `Matplotlib`.
- (c) Lorsque N double, par combien le temps d'exécution est-il multiplié ?
Concluez sur la complexité temporelle de cette opération et son impact sur la consommation énergétique de l'IA.

Chapitre 6

Exercices d'application

6.1 Calcul Différentiel et Optimisation

1. **Jacobienne de la fonction Softmax :**

Soit $z \in \mathbb{R}^K$ le vecteur des logits en entrée d'une couche Softmax. La i -ème sortie est définie par $\sigma(z)_i = \frac{e^{z_i}}{\sum_{k=1}^K e^{z_k}}$.

(a) Calculez la dérivée partielle $\frac{\partial \sigma(z)_i}{\partial z_j}$ dans le cas où $i = j$.

(b) Calculez la dérivée partielle $\frac{\partial \sigma(z)_i}{\partial z_j}$ dans le cas où $i \neq j$.

(c) Exprimez la matrice Jacobienne complète $J_\sigma(z) \in \mathbb{R}^{K \times K}$ sous forme compacte en utilisant le symbole de Kronecker δ_{ij} .

2. **Différentiation Numérique vs Analytique :**

Soit la fonction de perte $f(x, y) = x^2y + \sin(x) \cos(y)$.

(a) Écrivez une fonction Python `grad_exact(x, y)` qui renvoie le gradient analytique exact $\nabla f(x, y)$.

(b) Écrivez une fonction `grad_num(f, x, y, h=1e-5)` qui calcule le gradient approché par différences finies centrées : $\frac{\partial f}{\partial x} \approx \frac{f(x+h, y) - f(x-h, y)}{2h}$.

(c) Évaluez la norme de l'erreur $\|\nabla f_{\text{exact}} - \nabla f_{\text{num}}\|_2$ au point $(x = 2.0, y = -1.0)$. Concluez sur la précision du schéma.

3. **Régularisation L_2 (Ridge) et Vitesse de Convergence :**

Soit la fonction de coût régularisée $\mathcal{L}_R(w) = f(w) + \frac{\lambda}{2} \|w\|_2^2$, où f est L -lisse et m -strictement convexe.

(a) Calculez le gradient $\nabla \mathcal{L}_R(w)$ et la matrice Hessienne $\nabla^2 \mathcal{L}_R(w)$.

(b) Montrez que l'ajout du terme de régularisation $\lambda > 0$ augmente toutes les valeurs propres de la Hessienne de λ .

(c) Quel est l'impact de λ sur le nombre de conditionnement $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$? Pourquoi la régularisation L_2 accélère-t-elle la convergence de la descente de gradient ?

6.2 Algèbre Linéaire et Espaces Latents

4. **Égalité des spectres de $X^\top X$ et $XX^\top$:**

Soit $X \in \mathbb{R}^{N \times d}$ une matrice de données ($N > d$).

(a) Soit $\mathbf{v} \in \mathbb{R}^d$ un vecteur propre de $X^\top X$ associé à la valeur propre $\lambda \neq 0$. Posez $\mathbf{u} = X\mathbf{v}$. Montrez que $\mathbf{u}$ est un vecteur propre de $XX^\top$ associé à la même valeur propre λ .

- (b) Que pouvez-vous en conclure sur les valeurs singulières non nulles de la SVD de X ?
5. **Simulateur de mise à jour LoRA (Low-Rank Adaptation) :**
 En fine-tuning de LLM, au lieu de modifier une matrice $W_0 \in \mathbb{R}^{4096 \times 4096}$, on apprend $W = W_0 + \Delta W$ avec $\Delta W = A \cdot B$ où $A \in \mathbb{R}^{4096 \times 8}$ et $B \in \mathbb{R}^{8 \times 4096}$.
- (a) Sous NumPy, initialisez W_0 avec des valeurs gaussiennes, A aléatoirement et B avec des zéros (de sorte que $\Delta W = 0$ au départ).
- (b) Calculez la taille en octets de mémoire vive nécessaire pour stocker W_0 seule versus la taille des matrices A et B combinées (en précisant le type float32).
- (c) Calculez le rang de la matrice $A \cdot B$ via `np.linalg.matrix_rank`.
6. **Matrice de Projection Orthogonale :**
 Soit $X \in \mathbb{R}^{N \times p}$ de rang plein ($p < N$). La matrice $P = X(X^\top X)^{-1}X^\top \in \mathbb{R}^{N \times N}$ projette tout vecteur $y \in \mathbb{R}^N$ sur l'espace des colonnes de X .
- (a) Démontrez que P est symétrique ($P^\top = P$) et idempotente ($P^2 = P$).
- (b) Montrez que les seules valeurs propres possibles d'une matrice idempotente sont 0 et 1.
- (c) **(Code Python)** Créez une matrice X de dimension 100×2 . Calculez P . Générez un vecteur aléatoire $y \in \mathbb{R}^{100}$ et vérifiez numériquement que $P(Py) = Py$.

6.3 Probabilités, Inférence et Vraisemblance

7. **Estimation du Maximum de Vraisemblance (MLE) Gaussien :**
 Soient $x_1, \dots, x_N \sim \mathcal{N}(\mu, \sigma^2)$ des variables i.i.d.
- (a) Écrivez la fonction de Log-Vraisemblance $\ell(\mu, \sigma^2) = \log p(x_1, \dots, x_N | \mu, \sigma^2)$.
- (b) En annulant la dérivée partielle $\frac{\partial \ell}{\partial \mu}$, retrouvez l'estimateur du maximum de vraisemblance $\hat{\mu}_{\text{MLE}}$.
- (c) En annulant la dérivée partielle par rapport à σ^2 , montrez que $\hat{\sigma}_{\text{MLE}}^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{\mu})^2$. Cet estimateur est-il sans biais ?
8. **Classifieur Naïve Bayes "From Scratch" :**
 Soit un problème de classification binaire ($Y \in \{0, 1\}$) avec des caractéristiques continues $X = (X_1, X_2)$.
- (a) Créez un dataset synthétique de 200 points avec `make_blobs` de `sklearn`.
- (b) Calculez les moyennes $\mu_{k,j}$ et variances $\sigma_{k,j}^2$ pour chaque classe $k \in \{0, 1\}$ et chaque variable $j \in \{1, 2\}$.
- (c) Écrivez une fonction `predict_posterior(x_new)` qui calcule $\mathbb{P}(Y = 1 | X = \mathbf{x}_{\text{new}})$ en utilisant la formule de Bayes et l'hypothèse d'indépendance conditionnelle : $p(X | Y) = p(X_1 | Y) \cdot p(X_2 | Y)$.
9. **Divergence de Kullback-Leibler (KL-Divergence) :**
 La divergence KL entre deux lois P et Q mesure leur "distance" en théorie de l'information : $D_{\text{KL}}(P \parallel Q) = \int P(x) \log \frac{P(x)}{Q(x)} dx$.

- (a) Démontrez analytiquement que pour deux gaussiennes 1D $P \sim \mathcal{N}(\mu_1, \sigma_1^2)$ et $Q \sim \mathcal{N}(\mu_2, \sigma_2^2)$, la divergence KL vaut :

$$D_{\text{KL}}(P \parallel Q) = \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}$$

- (b) **(Code Python)** Écrivez une fonction Python calculant cette divergence et vérifiez que $D_{\text{KL}}(P \parallel Q) \geq 0$ pour plusieurs choix de paramètres.

6.4 Statistiques, EDA et Évaluation

10. Décomposition Biais-Variance :

Soit $y = f(x) + \varepsilon$ avec $\mathbb{E}[\varepsilon] = 0$ et $\text{Var}(\varepsilon) = \sigma^2$. Soit $\hat{f}(x)$ la prédiction d'un modèle entraîné sur un jeu de données aléatoire. Démontrez la décomposition classique de l'erreur quadratique moyenne attendue :

$$\mathbb{E}_{\mathcal{D}} \left[(y - \hat{f}(x))^2 \right] = \text{Biais}(\hat{f}(x))^2 + \text{Var}(\hat{f}(x)) + \sigma^2$$

(Indice : ajoutez et soustrayez $\mathbb{E}[\hat{f}(x)]$ à l'intérieur du carré).

11. Intervalle de Confiance par Bootstrap d'un Score F1 :

Sur un jeu de test de $N = 100$ exemples, un modèle obtient un F1-score de 0.82.

- Simulez un tableau NumPy de prédictions binomiales et de vraies étiquettes de taille 100.
- Implémentez une boucle de Bootstrap ($B = 5000$ rééchantillonnages avec remise) pour calculer une distribution de F1-scores empiriques.
- Extrayez les percentiles à 2.5% et 97.5% pour donner l'intervalle de confiance à 95% du F1-score de l'IA.

12. Détection de Data Drift via le test Kolmogorov-Smirnov :

Lorsqu'une IA est déployée, la distribution des données réelles $P_{\text{prod}}(X)$ peut dériver de la distribution d'entraînement $P_{\text{train}}(X)$.

- (Théorie)** Le test K-S compare la fonction de répartition empirique $F_1(x)$ et $F_2(x)$. Quelle est sa statistique de test D ?
- (Code Python)** Générez $X_{\text{train}} \sim \mathcal{N}(0, 1)$ ($N = 1000$) et $X_{\text{prod}} \sim \mathcal{N}(0.2, 1.1)$ ($N = 500$).
- Utilisez `scipy.stats.ks_2samp` pour tester s'il y a un Data Drift. Interprétez la p -valeur obtenue face à un seuil $\alpha = 0.01$.

6.5 Mathématiques Discrètes, Graphes et Complexité

13. Combinatoire d'un Arbre de Décision :

Soit un dataset de N exemples décrits par M variables continues.

- Pour une variable continue donnée, combien de seuils de coupure (*splits*) uniques et pertinents peut-on tester au maximum ?
- En déduire le nombre total de décisions possibles que l'algorithme d'arbre de décision doit évaluer à la racine pour choisir la première séparation.

14. Tri Topologique d'un Graphe d'Autograd :

Un graphe de calcul est représenté par le dictionnaire d'adjacence suivant : `graph = {'x': ['mult'], 'W': ['mult'], 'mult': ['add'], 'b': ['add'], 'add': ['loss']}`.

- Construisez ce graphe orienté à l'aide de `networkx.DiGraph`.
- Écrivez une fonction de parcours en profondeur (DFS) ou utilisez `nx.topological_sort` pour obtenir l'ordre d'exécution exact de la passe avant (*Forward Pass*).
- Inversez cette liste pour obtenir l'ordre d'exécution de la passe arrière (*Backward Pass / Backpropagation*).

15. Matrice d'Adjacence et Nombre de Chemins :

Soit $A \in \{0, 1\}^{n \times n}$ la matrice d'adjacence d'un graphe non orienté G .

- Démontrez par récurrence sur $k \geq 1$ que le coefficient $(A^k)_{ij}$ donne exactement le nombre de chemins de longueur k reliant le sommet i au sommet j .
- (Code Python)** Soit un réseau social à 5 personnes. Créez sa matrice d'adjacence sous NumPy, calculez A^3 et déduisez-en le nombre d'intermédiaires à 2 saut(s) entre chaque paire d'utilisateurs.

6.6 Exercices Intégratifs de Synthèse (Multi-Chapitres)

16. Gradient de la Factorisation Matricielle :

Dans les systèmes de recommandation, on approxime une matrice de notes $R \in \mathbb{R}^{m \times n}$ par le produit de deux matrices de rang faible $U \in \mathbb{R}^{m \times k}$ et $V \in \mathbb{R}^{n \times k}$ en minimisant la perte de Frobenius :

$$\mathcal{L}(U, V) = \|R - UV^\top\|_F^2 = \text{tr} \left((R - UV^\top)(R - UV^\top)^\top \right)$$

- En utilisant les règles de dérivée matricielle et de la trace, démontrez que $\nabla_U \mathcal{L} = -2(R - UV^\top)V$.
 - Démontrez de même que $\nabla_V \mathcal{L} = -2(R - UV^\top)^\top U$.
 - Écrivez la règle de mise à jour des matrices U et V pour un pas d'apprentissage α .
- 17. Loi Gaussienne Multivariée et Distance de Mahalanobis :**
La densité d'un vecteur aléatoire gaussien $X \in \mathbb{R}^d$ est

$$f(x) = \frac{1}{(2\pi)^{d/2} \det(\Sigma)^{1/2}} \exp \left(-\frac{1}{2} (x - \mu)^\top \Sigma^{-1} (x - \mu) \right)$$

- Que représente la quantité $D_M(x) = \sqrt{(x - \mu)^\top \Sigma^{-1} (x - \mu)}$?
 - Expliquez géométriquement pourquoi la distance de Mahalanobis prend en compte la covariance des données, contrairement à la simple distance euclidienne $\|x - \mu\|_2$.
 - Si l'on diagonalise $\Sigma = Q\Lambda Q^\top$, montrez que $D_M(x)^2$ se ramène à une somme de carrés de variables décorréliées et réduites.
- 18. Régression Logistique de A à Z :**
Soit un modèle de classification binaire $p = \sigma(w^\top x)$.
- Montrez que la Log-Vraisemblance Négative (NLL) pour N observations est la perte d'Entropie Croisée :

$$\mathcal{L}(w) = - \sum_{i=1}^N [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

- (b) Calculez le gradient $\nabla_w \mathcal{L}(w)$ et montrez qu'il s'écrit sous forme matricielle très pure : $X^\top (\mathbf{p} - \mathbf{y})$.
- (c) Expliquez comment la matrice Hessienne $\nabla_w^2 \mathcal{L} = X^\top S X$ (où S est diagonale) permet de calculer l'écart-type des poids appris et de réaliser un test de Wald sur l'importance des variables.

19. Le Laplacien de Graphe et les GNNs :

Soit un graphe $G = (V, E)$ de matrice d'adjacence A et de matrice des degrés D (diagonale avec $D_{ii} = \deg(i)$). La matrice Laplacienne est définie par $L = D - A$.

- (a) Pour un vecteur de signal $f \in \mathbb{R}^{|V|}$ qui associe une valeur à chaque sommet, montrez la formule quadratique fondamentale :

$$f^\top L f = \frac{1}{2} \sum_{\{i,j\} \in E} (f_i - f_j)^2$$

- (b) En déduire que L est une matrice symétrique semi-définie positive.
- (c) **(IA/GNN)** Pourquoi la valeur propre $\lambda_1 = 0$ est-elle toujours présente? Que mesure la seconde plus petite valeur propre λ_2 (connectivité de Fiedler) en termes de découpage de réseaux de neurones sur graphes?

20. La Perplexité des Modèles de Langage (LLM) :

Lors de l'évaluation d'un modèle comme GPT, la métrique standard est la **Perplexité** (*Perplexity* ou PPL).

- (a) Soit une séquence de N tokens $W = (w_1, w_2, \dots, w_N)$. La Perplexité est définie par :

$$\text{PPL}(W) = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i \mid w_1, \dots, w_{i-1}) \right)$$

Liez cette formule à l'Entropie Croisée moyenne par token ($\bar{H}$).

- (b) Si le modèle hésite uniformément entre K mots équiprobables à chaque étape (incertitude maximale), que vaut la Perplexité?
- (c) Interprétez concrètement ce que signifie une Perplexité de 15 pour un LLM dans la génération du prochain mot d'un texte.